Logistic Regression Classification metrics

Vladislav Goncharenko
ML researcher

girafe ai

ITMO, fall 2026

Recap

Lecture 2:
Linear Regression



1. Regression model
2. Linear models overview
3. Linear regression
    a. MSE loss
4. Gauss-Markov theorem
    a. BLUE
5. Regularizations
    a. L2, L1, ElasticNet
6. Model validation and evaluation
    a. Parameters and hyperparameters
    b. Dataset splits
    c. Stages organization

Outline

Lecture 3:
Logistic Regression
Classification metrics



1. Linear classification
    a. margin
    b. loss functions
2. Logistic Regression
    a. sigmoid derivation
    b. MLE for LogReg
    c. logistic loss
    d. probability calibration
3. Multiclass aggregation strategies
    a. One vs Rest
    b. One vs One
4. Metrics in classification
    a. Accuracy, Balanced accuracy
    b. Precision, Recall, F-score
    c. ROC curve, PR curve, AUC
    d. Confusion matrix
5. Hardware for ML

Linear Classification

girafe ai

01

Classification problem statement

$$X \in R^{n \times p}$$

$$Y \in C^n$$

$$|C| < +\infty$$

$$\text{e.g. } C = \{-1, 1\}$$

$$c(X) = \hat{Y} \approx Y$$

Variants of labels

- Binary - choice between two classes, easier for theoretical analysis also widely used in practice
  - loan return state
- Multiclass - choose one of many classes
  - spies of an animal
- Multilabel - choose several of many classes
  - genre of a film

Linear classifier

The most simple linear classifier

$$c(x) = \begin{cases} 1, & \text{if } f(x) \geq 0 \\ -1, & \text{if } f(x) < 0 \end{cases}$$

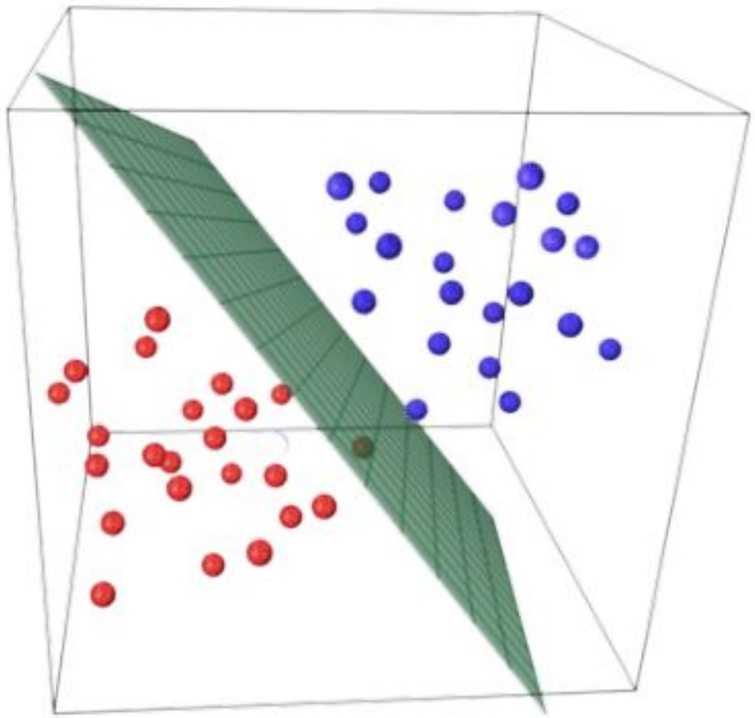
or equivalently

$$c(x) = \text{sign}(f(x)) = \text{sign}(x^T w)$$

Why cutoff value is fixed?

Bias term is implied

Geometrical interpretation



Geometrical interpretation:
hyperplane dividing space into two subspaces.

What is geometrical interpretation for parameters vector w?

$$c(x) = \text{sign}(f(x)) = \text{sign}(x^T w)$$

Margin

Let’s define binary linear model’s Margin as

$$M_i = y_i \cdot f(x_i) = y_i \cdot x_i^T w$$

main property:

negative margin reveals misclassification

$$M_i > 0 \Leftrightarrow y_i = c(x_i)$$

$$M_i \leq 0 \Leftrightarrow y_i \neq c(x_i)$$

Margin is used in theoretical proofs

Weights choice intuition

Remembering old paradigm

Empirical risk = $\sum_{\text{by object}}$ Loss on object $\longrightarrow$ Min model params

Essential error is misclassification

$$L_{\text{mis}}(y_i^t, y_i^p) = [y_i^t \neq y_i^p] = [M_i \leq 0]$$

Disadvantages:

- Not differentiable
- Overlooks confidence

Iverson bracket $[P] = \begin{cases} 1, & \text{if } P \text{ is true} \\ 0, & \text{otherwise} \end{cases}$

Solution:

estimate it with a smooth function

Square loss

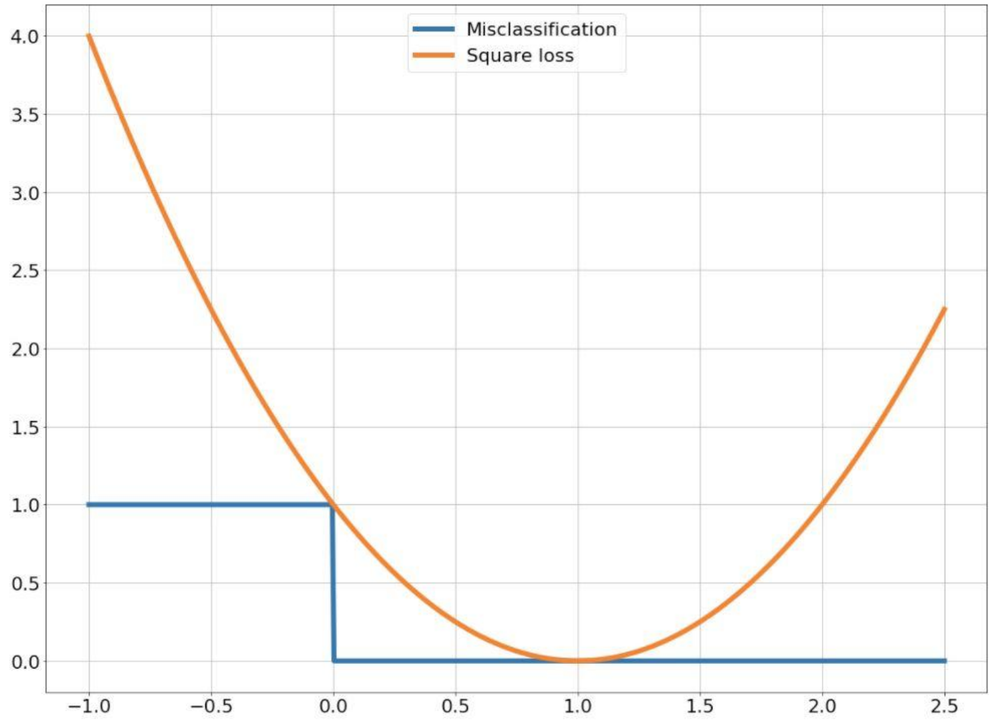
Let’s treat classification problem as regression problem:

thus we optimize MSE

$$Y \in \{-1, 1\} \mapsto Y \in R$$

$$L_{\text{MSE}} = (y_i - x_i^T w)^2 = \frac{(y_i^2 - y_i \cdot x_i^T w)^2}{y_i^2} =$$

$$= (1 - y_i \cdot x_i^T w)^2 = (1 - M_i)^2$$

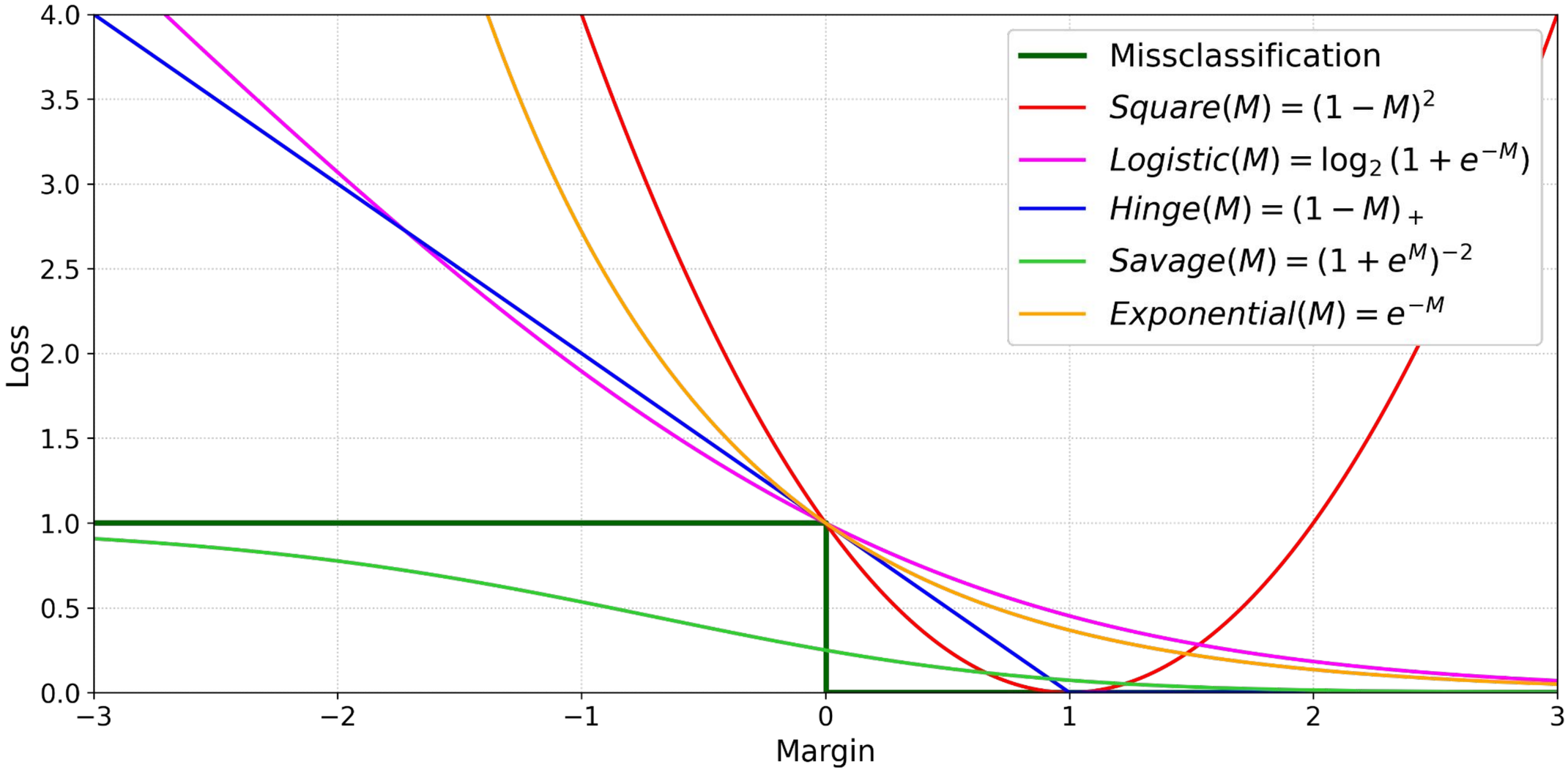


Advantage: already solved

Disadvantage: penalizes for high confidence

No one uses it in practice!

Variety of losses motivation



Loss functions for classification

Logistic Regression

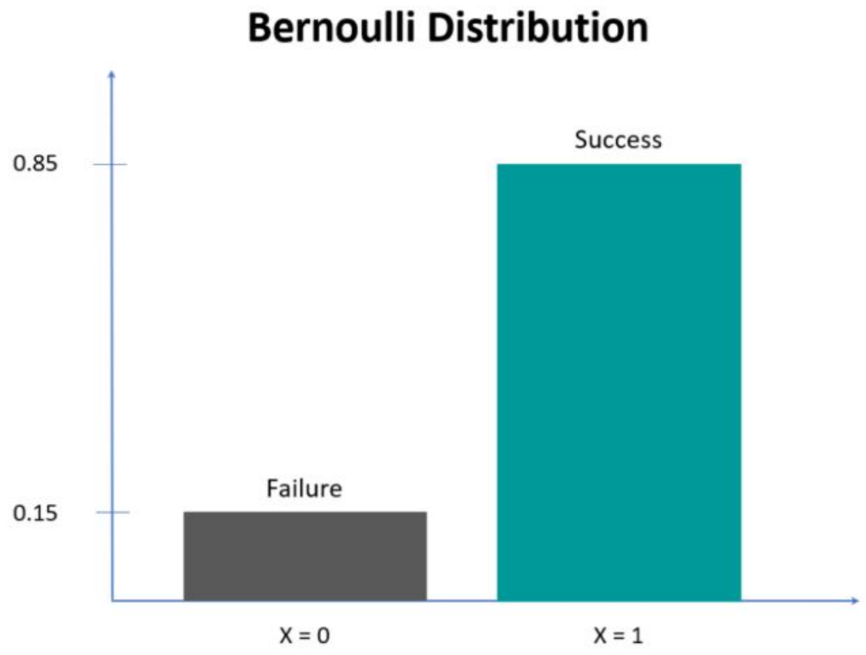
girafe ai

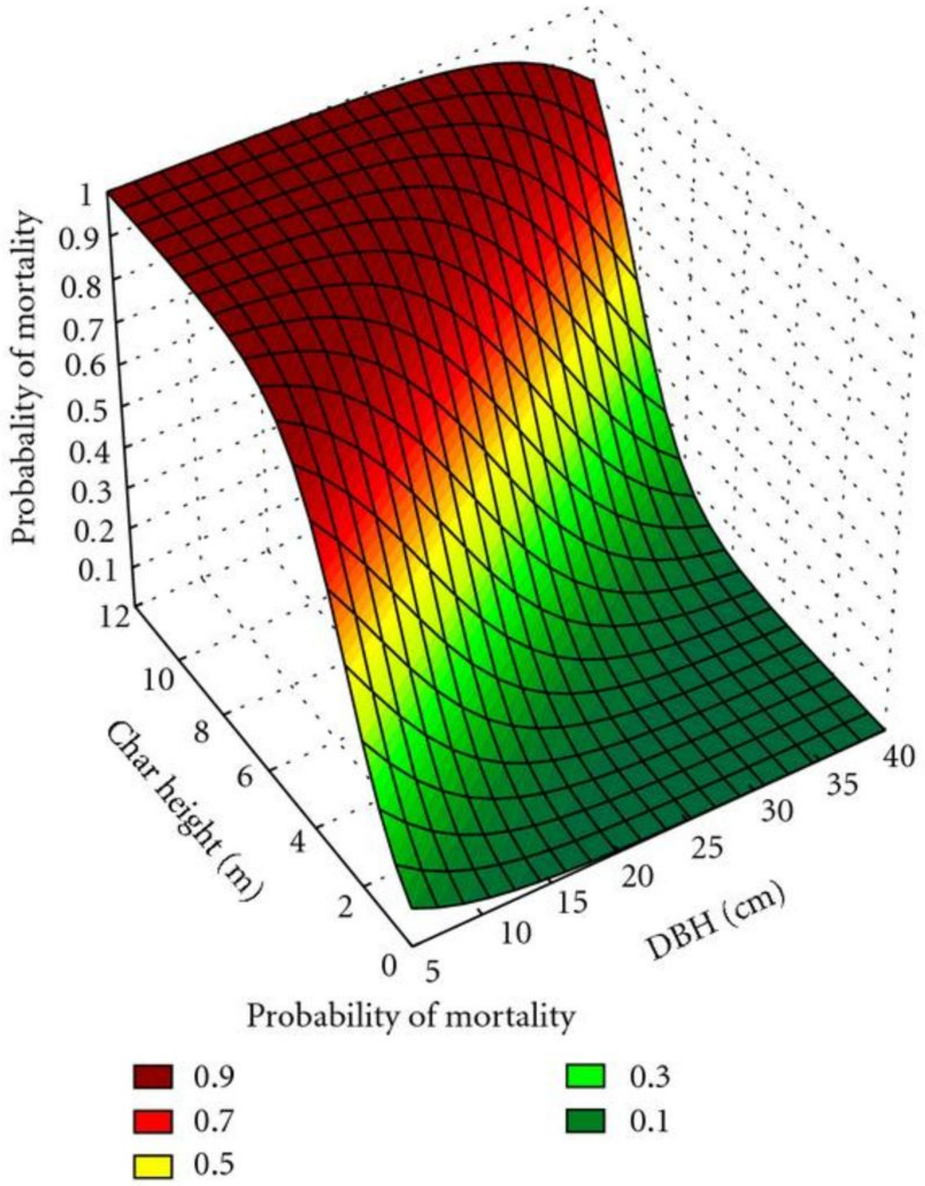
02

Basic assumption

of Logistic regression is that every point of feature space have associated probability of being certain class.

So LogReg generates Bernoulli distribution for each point (since we have two classes).





Intuition

I. Let's try to predict probability of an object to have positive class

$$p_+ = P(y = 1|x) \in [0, 1]$$

II. But all we can predict is a real number!

$$y = x^T w \in R$$

III. Time for some tricks

$$\frac{p_+}{1 - p_+} \in [0, +\infty)$$

$$\log \frac{p_+}{1 - p_+} \in R$$

Here is the match

This is called **logit** or **log-odds**

IV. Reverse to closed form

$$\frac{p_+}{1 - p_+} = \exp(x^T w)$$

$$p_+ = \frac{1}{1 + exp(-x^T w)} = \sigma(x^T w)$$

Sigmoid (aka logistic) function

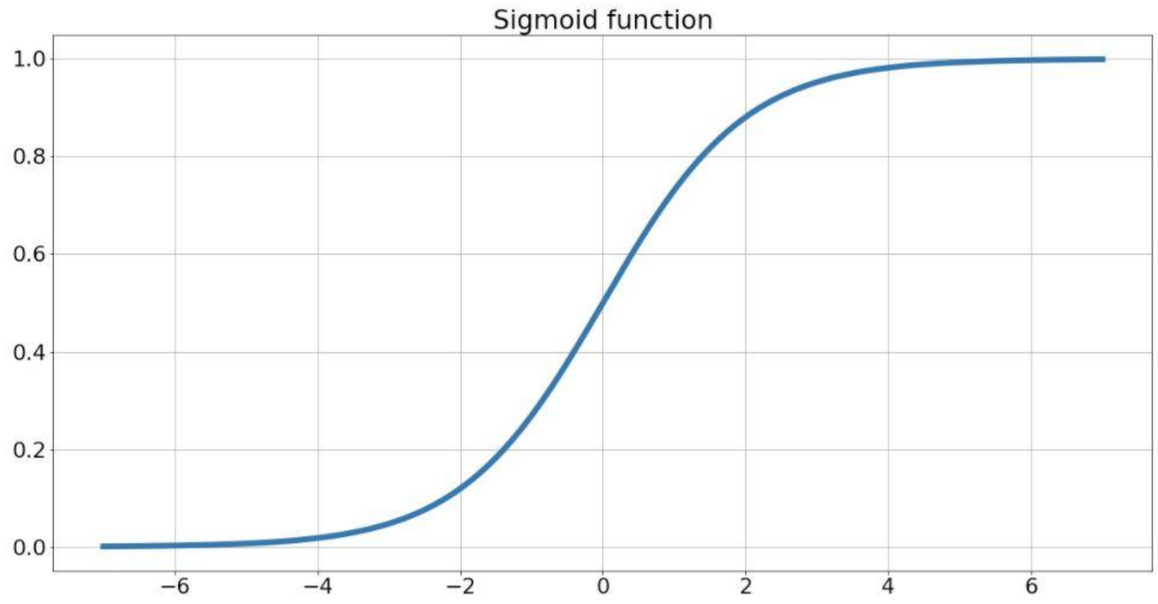
$$\sigma(x) = \frac{1}{1 + exp(-x)}$$

Sigmoid is odd relative to (0, 0.5) point

Symmetric property:

$$1 - \sigma(x) = \sigma(-x)$$

p+



Logit

Derivative: $$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

MLE for Logistic Regression

Just to remind

$$\log L(w|X,Y) = \log P(X,Y|w) = \log \prod_{i=1}^{n} P(x_i, y_i|w)$$

Calculating probabilities for objects (which are modelled as Bernoulli variables)

$$\begin{aligned} \text{if } y_i = 1: &\quad P(x_i, 1|w) = \sigma_w(x_i) = \sigma_w(M_i) \\ \text{if } y_i = -1: &\quad P(x_i, -1|w) = 1 - \sigma_w(x_i) = \sigma_w(-x_i) = \sigma_w(M_i) \end{aligned}$$

$$\log L(w|X,Y) = \sum_{i=1}^{n} \log \sigma_w(M_i) = -\sum_{i=1}^{n} \log(1 + \exp(-M_i)) \to \min_w$$

Proof

Problem flow

Problem
Loss; metrics
Dataset
Models
params

Problem flow



Target
Prediction

MLE
target θ

-> Loss -> Optimization methods
Gradient based

Root of the name

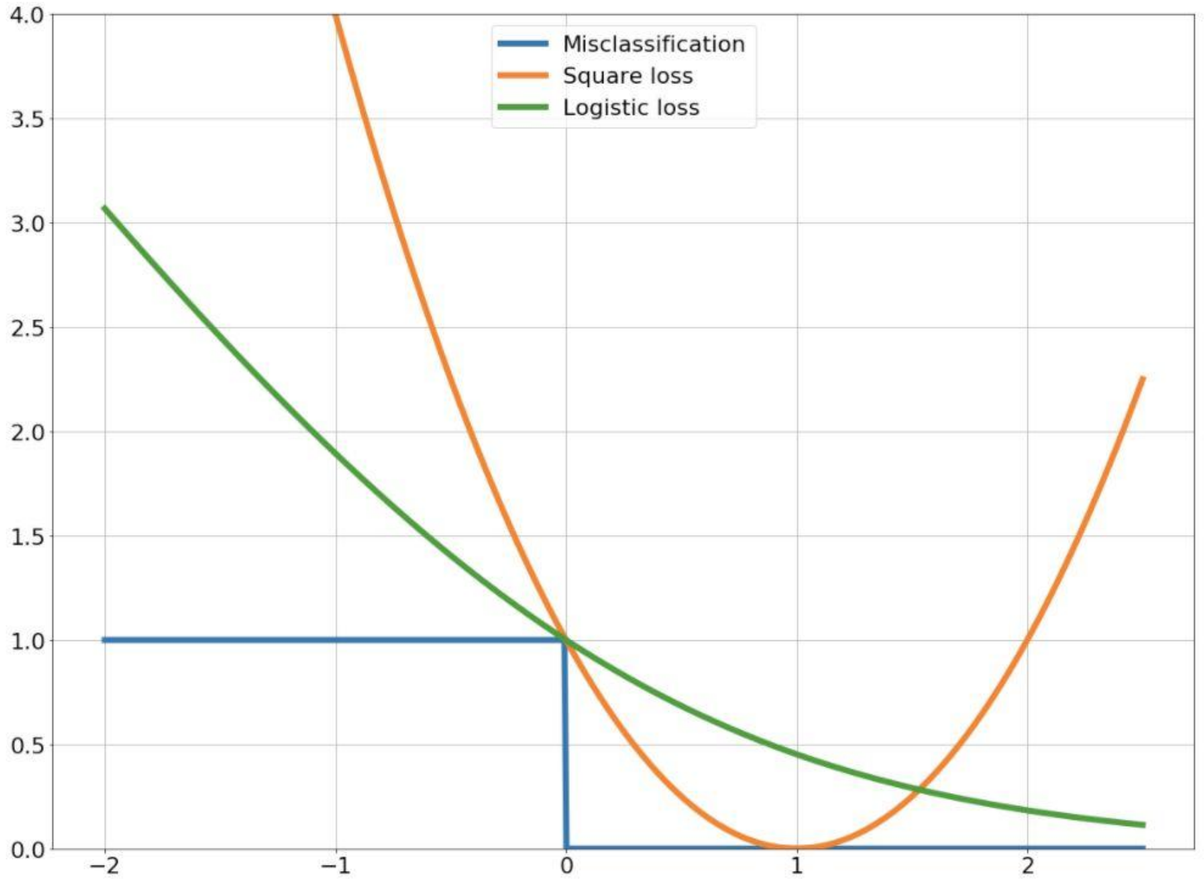
- Logistic - from logarithm in the loss function
- Regression - from the fact that we predict probability
  - which is continuous function of input





Logistic loss

$$L_{Logistic} = \log(1 + \exp(-M_i))$$

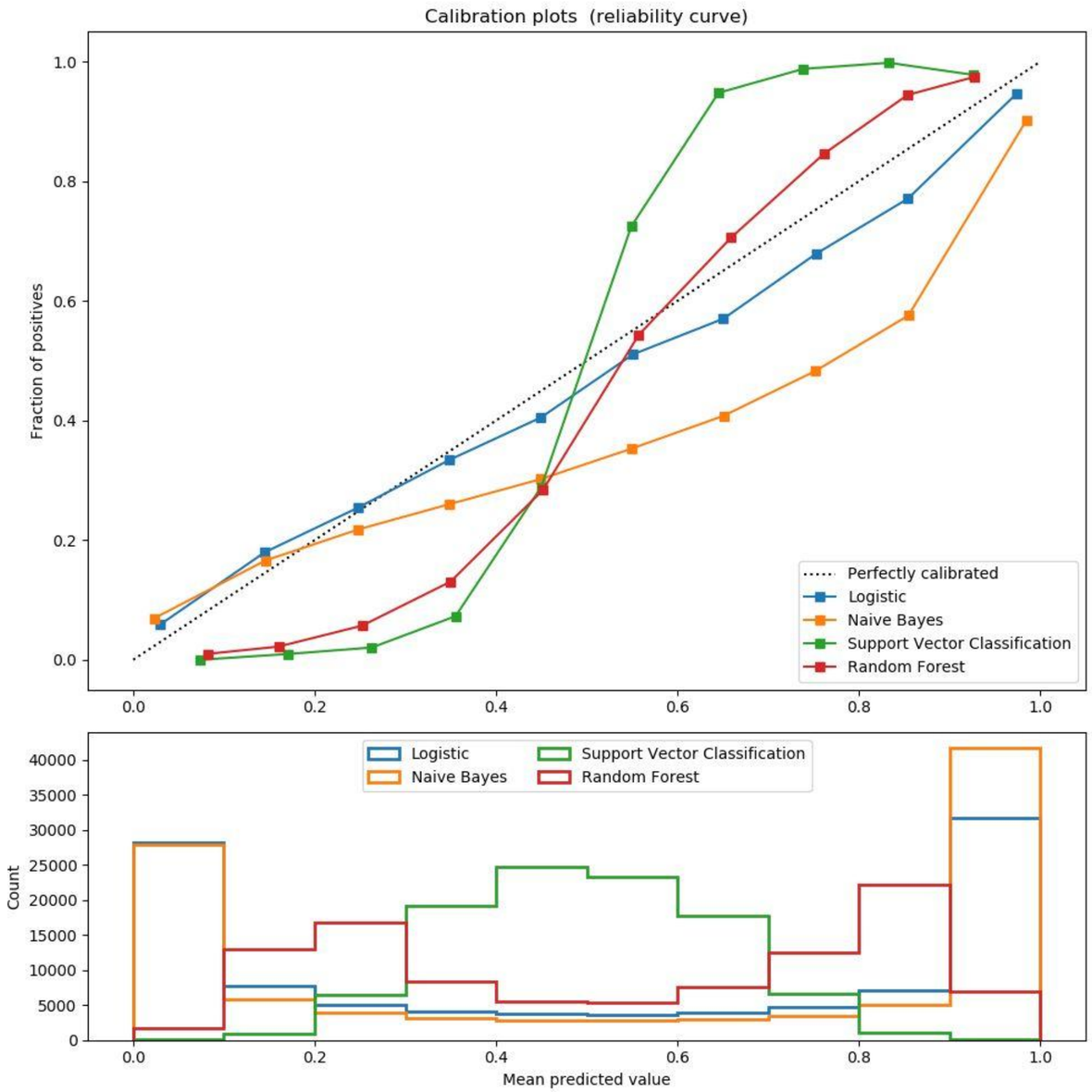


Probability calibration

By using Logistic Regression
we generate a Bernoulli distribution
in each point of space, so in practice it
means calibrated classifier

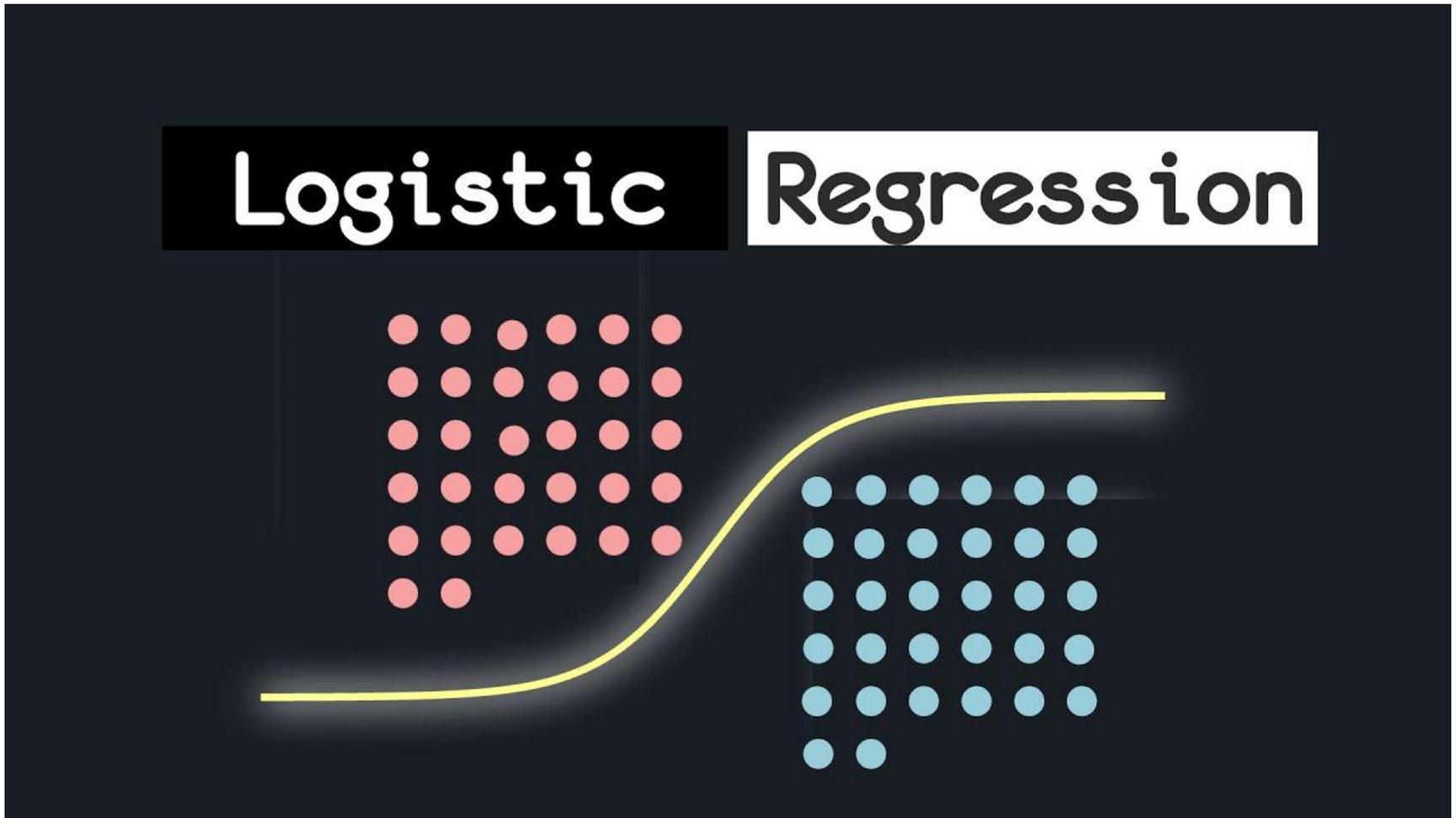
[Calibration discussion]

[Video overview]



Simple LR explanation

Logistic Regression (and why it's different from Linear Regression)
https://www.youtube.com/watch?v=3bvM3NyMiE0



Generalized linear model

Common distributions with typical uses and canonical link functions

Distribution	Support of distribution	Typical uses	Link name	Link function, $\mathbf{X}\boldsymbol{\beta} = g(\mu)$	Mean function
Normal	real: $(-\infty, +\infty)$	Linear-response data	Identity	$\mathbf{X}\boldsymbol{\beta} = \mu$	$\mu = \mathbf{X}\boldsymbol{\beta}$
Exponential	real: $(0, +\infty)$	Exponential-response data, scale parameters	Negative inverse	$\mathbf{X}\boldsymbol{\beta} = -\mu^{-1}$	$\mu = -(\mathbf{X}\boldsymbol{\beta})^{-1}$
Gamma					
Inverse Gaussian	real: $(0, +\infty)$		Inverse squared	$\mathbf{X}\boldsymbol{\beta} = \mu^{-2}$	$\mu = (\mathbf{X}\boldsymbol{\beta})^{-1/2}$
Poisson	integer: $0, 1, 2, \ldots$	count of occurrences in fixed amount of time/space	Log	$\mathbf{X}\boldsymbol{\beta} = \ln(\mu)$	$\mu = \exp(\mathbf{X}\boldsymbol{\beta})$
Bernoulli	integer: $\{0, 1\}$	outcome of single yes/no occurrence		$\mathbf{X}\boldsymbol{\beta} = \ln\left(\frac{\mu}{1-\mu}\right)$	
Binomial	integer: $0, 1, \ldots, N$	count of # of "yes" occurrences out of N yes/no		$\mathbf{X}\boldsymbol{\beta} = \ln\left(\frac{\mu}{n-\mu}\right)$	

Depending on which distribution we want to predict we have different forms of parameters encoded to formula.

See https://en.wikipedia.org/wiki/Generalized_linear_model#Link_function

Video on GLM

Exponential family of functions

$$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta))$$

where

- $T(y)$ — sufficient statistics
- η — natural parameters
- $a(\eta)$ — log-partition function (normalizer)
- $b(y)$ — base measure

Exponential families arise naturally as the answer to the question:
what is the **maximum-entropy distribution** consistent with given constraints on expected values?

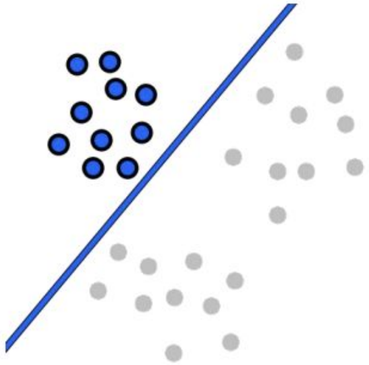
https://en.wikipedia.org/wiki/Exponential_family

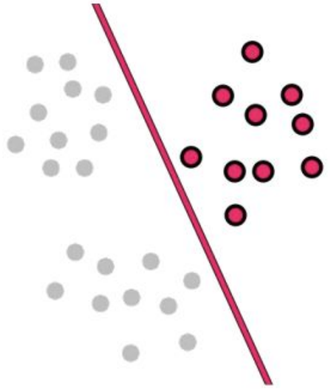
Multiclass aggregation strategies

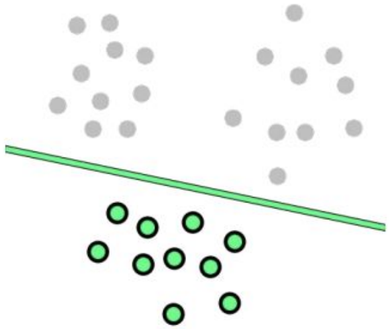
girafe ai

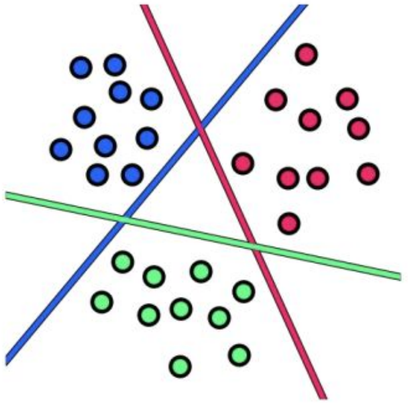
03

One vs Rest training

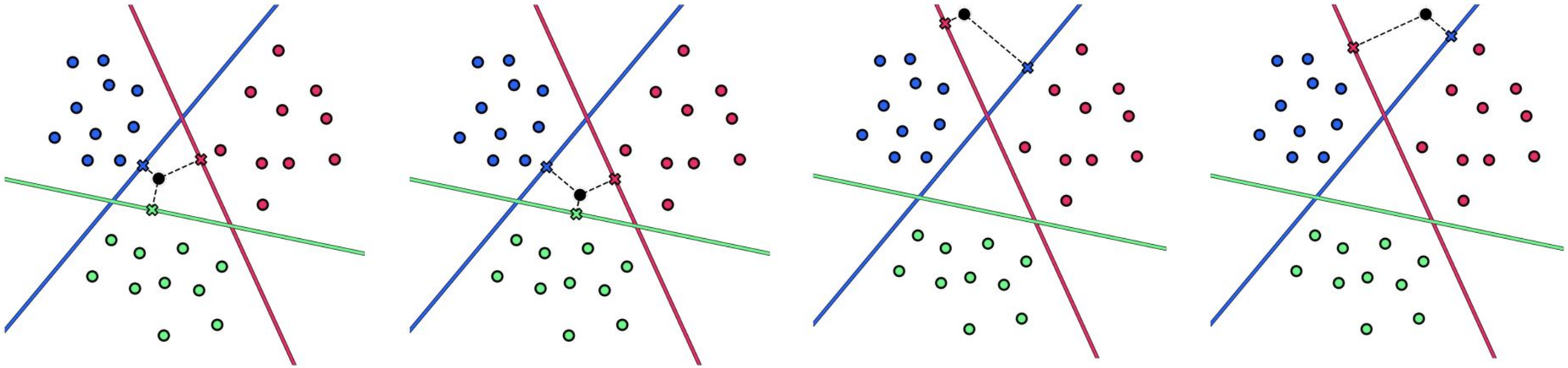






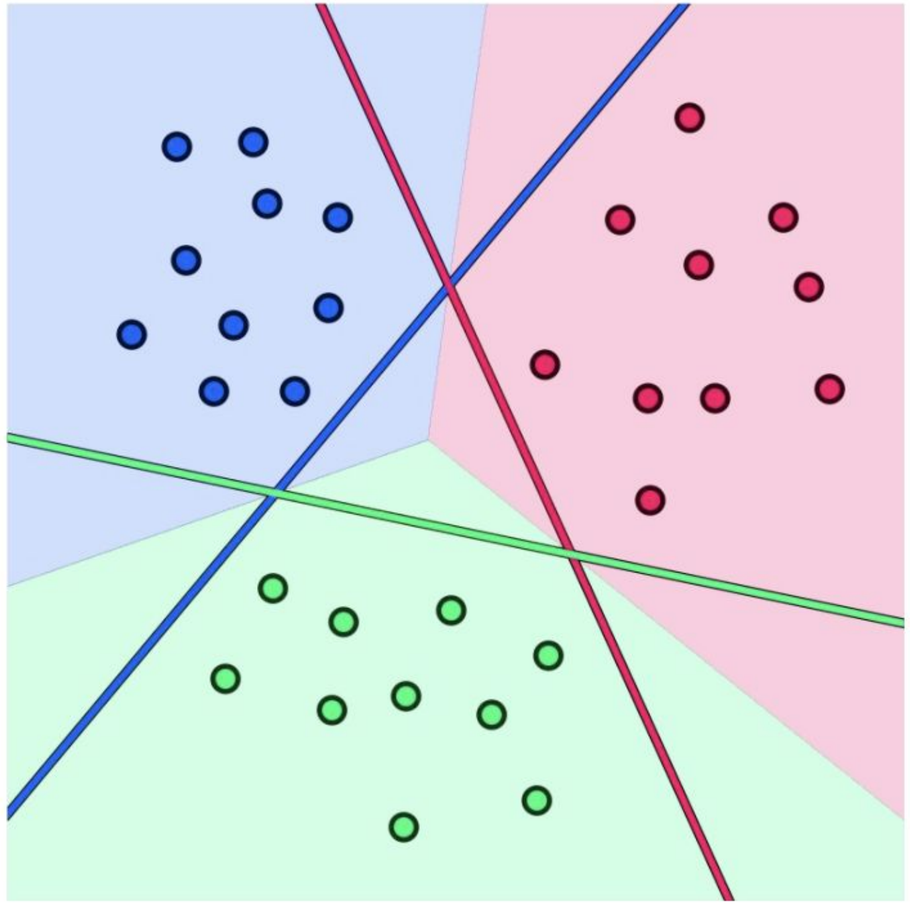


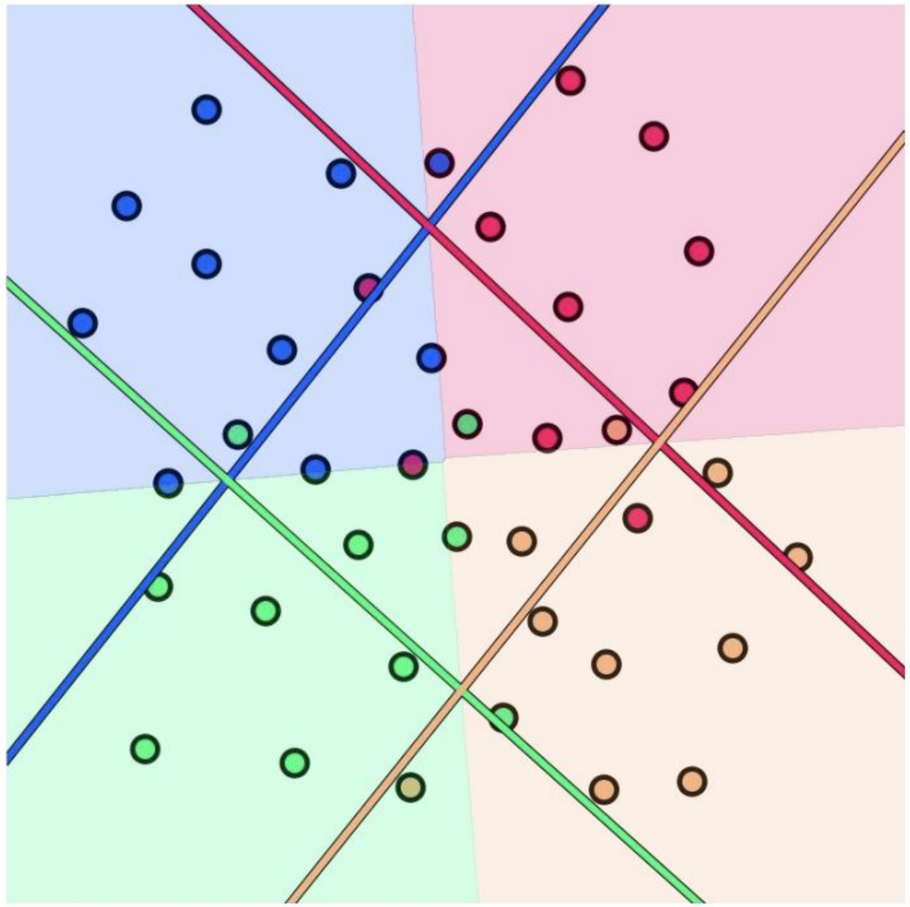
One vs Rest inference algorithm



Take class with max margin

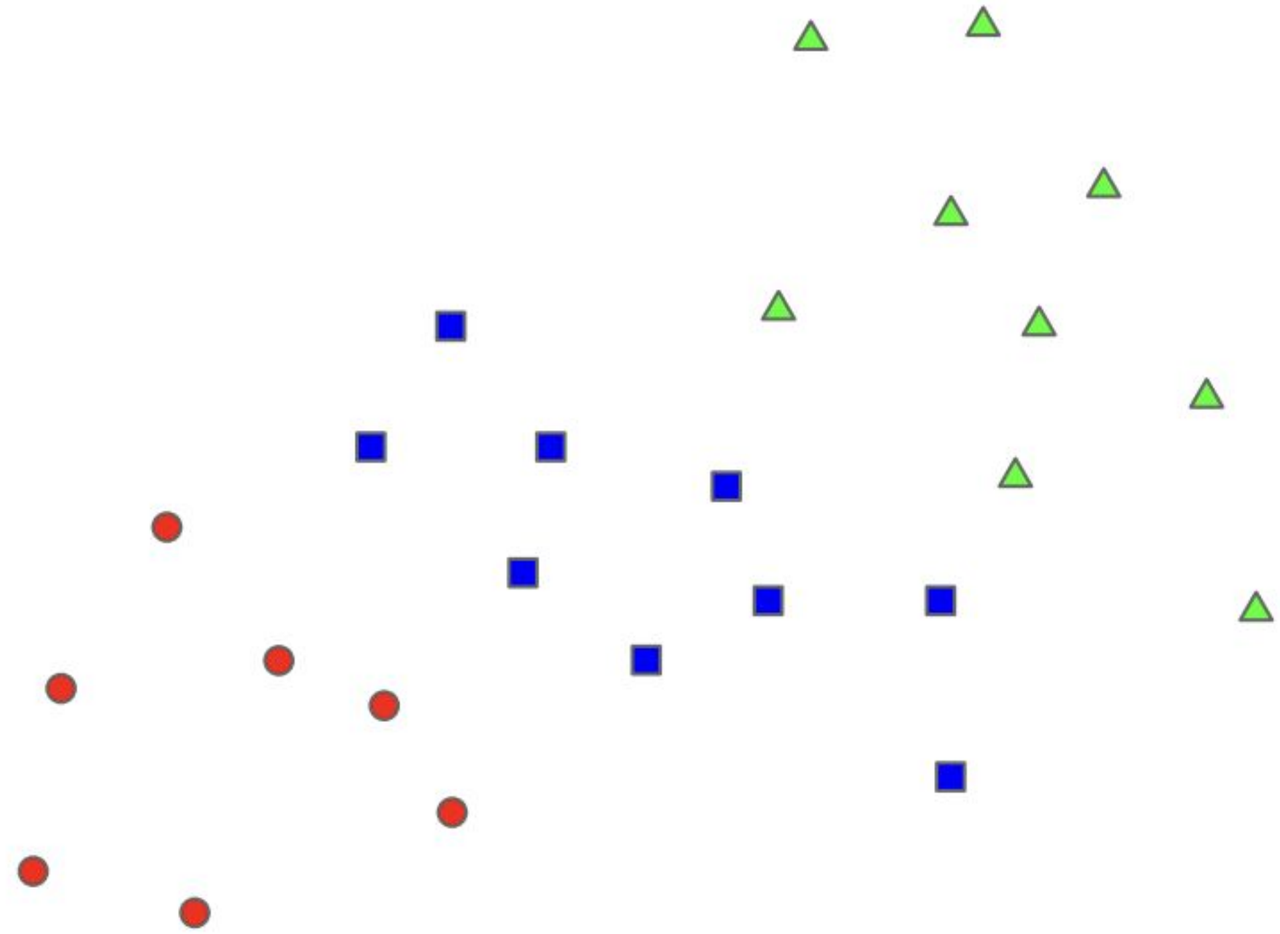
One vs Rest final result





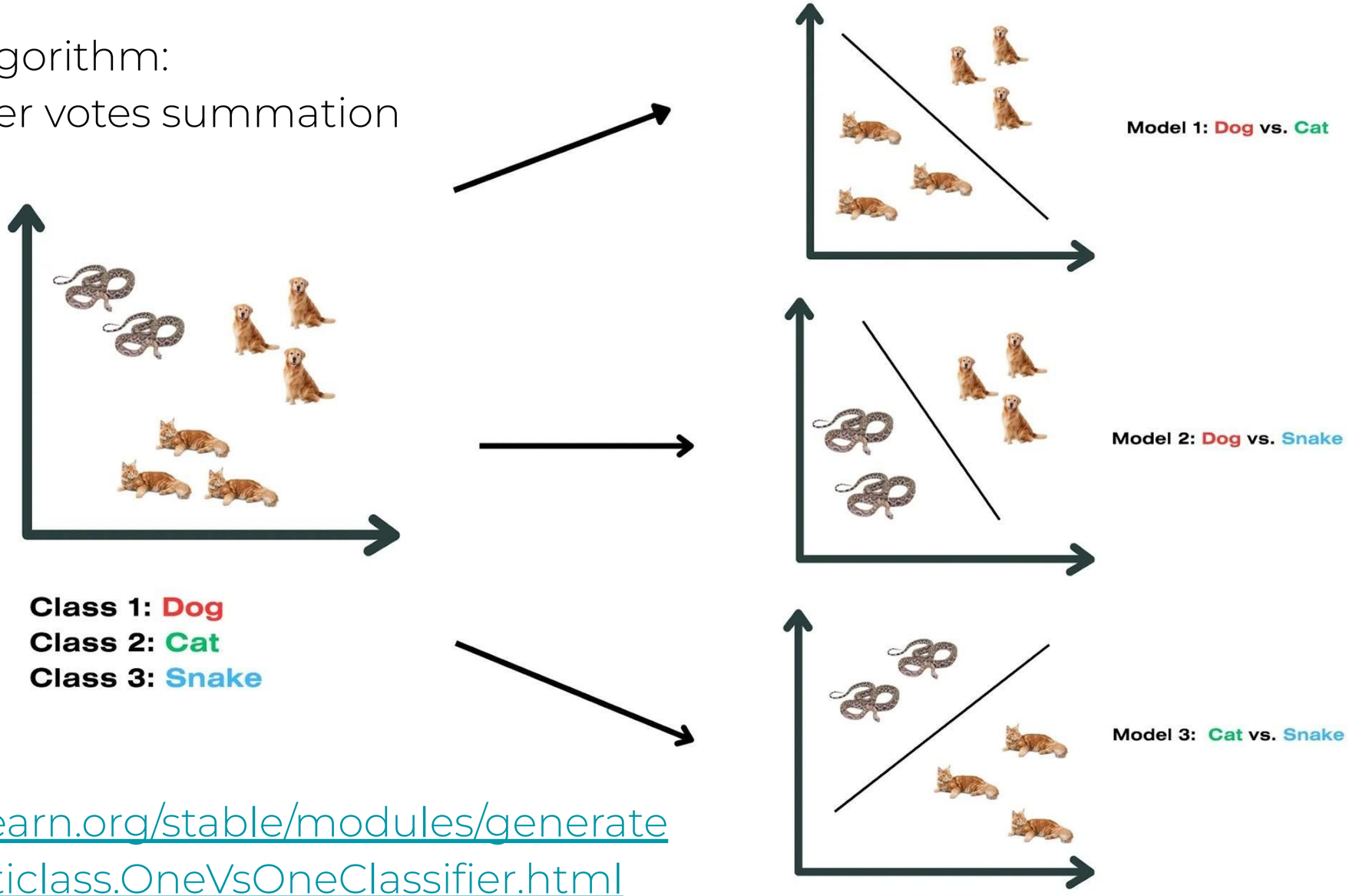
https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html

One vs Rest Failure case?



One vs One

Inference algorithm:
base classifier votes summation

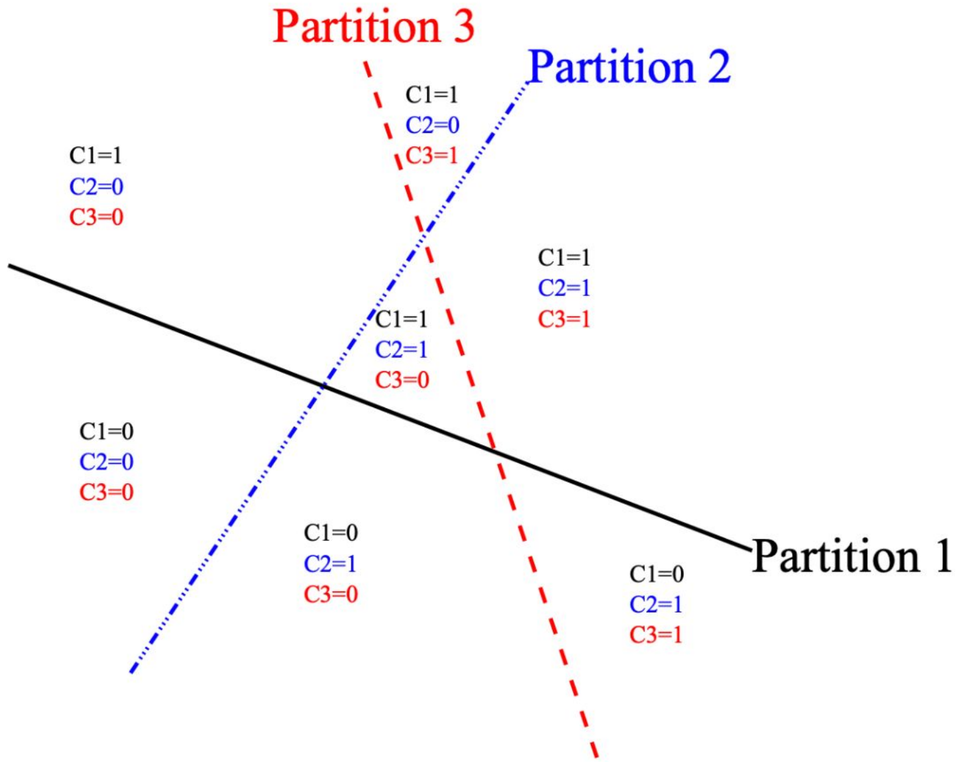


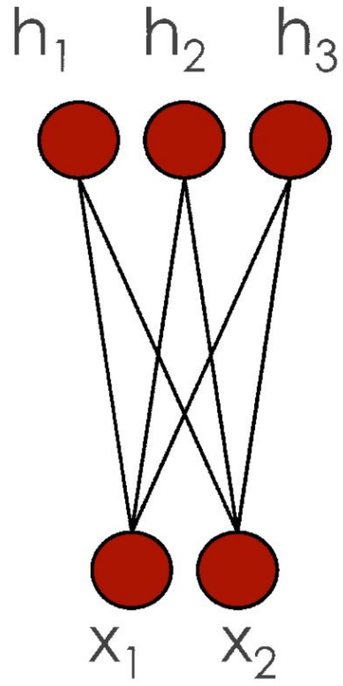
https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsOneClassifier.html

Summary

	One vs Rest	One vs One
#classifiers	k	k(k-1)/2
dataset for each	full	subsampled

The twist


Partition 3
Partition 2
Partition 1
C1=1
C2=0
C3=0
C1=1
C2=0
C3=1
C1=1
C2=1
C3=1
C1=1
C2=1
C3=0
C1=0
C2=0
C3=0
C1=0
C2=1
C3=0
C1=0
C2=1
C3=1


h1
h2
h3
x1
x2

Metrics in classification

girafe ai

04

Metric definition

Metric is a function of model prediction on objects and true answers which shows how good (or bad) these predictions are.

Best solution gives extremum of metric.

Loss and metric comparison

	Loss	Metric
Direction	↓ Down	↑ Up
Differentiability	Yes	No
Limits	R	[0, 1]
Used	Train	Val, Test
Interpretability	No	Tries
Count	1	Many

Main classification metrics



- Accuracy
  - Balanced accuracy
- Confusion matrix
- Precision, Recall and friends
- F-score
- ROC curve
  - ROC-AUC
- PR curve
  - PR-AUC
- Multiclass generalizations

Accuracy

Ratio of right classifications

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n}[y_i^t = y_i^p]$$

target: 1 0 1 0 0 0 0 1 0 0

predicted: 0 0 1 0 0 0 0 1 1 0

accuracy = 8/10 = 0.8

$$\text{Balanced accuracy} = \frac{1}{C}\sum_{k=1}^{C}\frac{\sum_i[y_i^t = k \text{ and } y_i^t = y_i^p]}{\sum_i[y_i^t = k]}$$

Confusion matrix

		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive,** Type I error
	Predicted condition negative	**False negative,** Type II error	**True negative**

https://en.wikipedia.org/wiki/Confusion_matrix

Accuracy formula

What is formula for accuracy in terms of these categories?

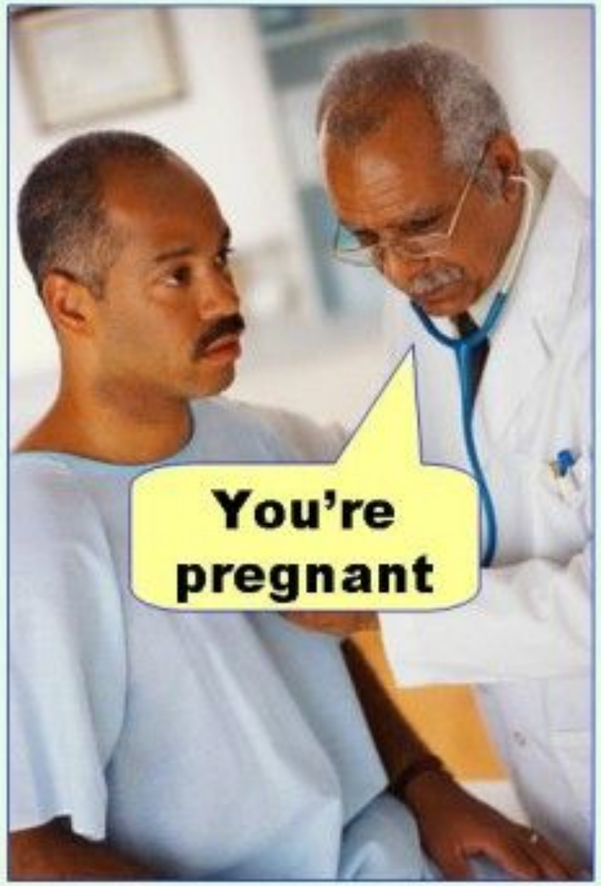
		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

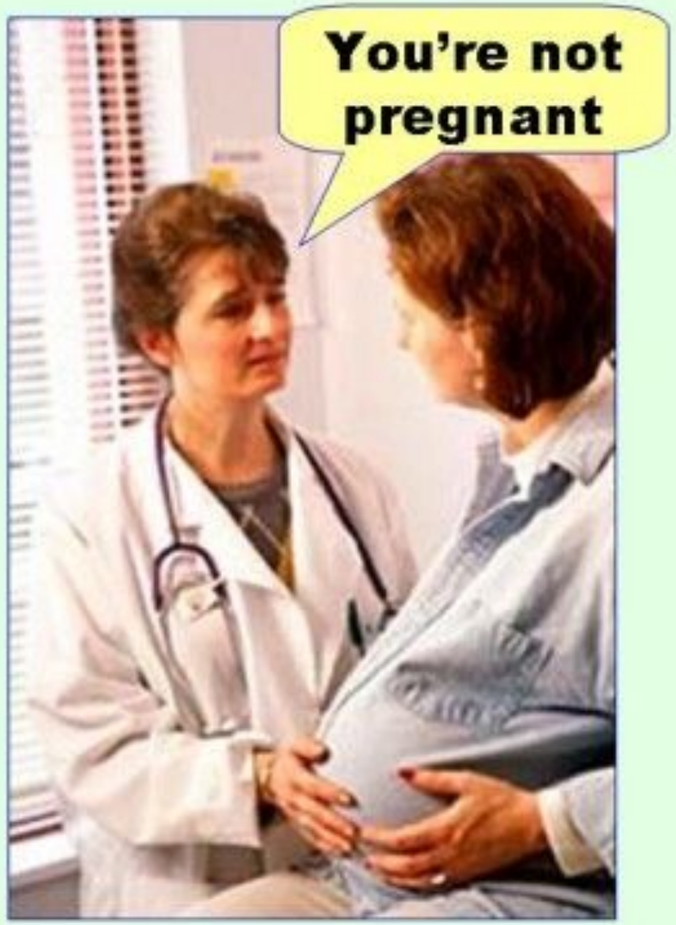
Types of errors

Positive condition is being pregnant as usual in medical tests

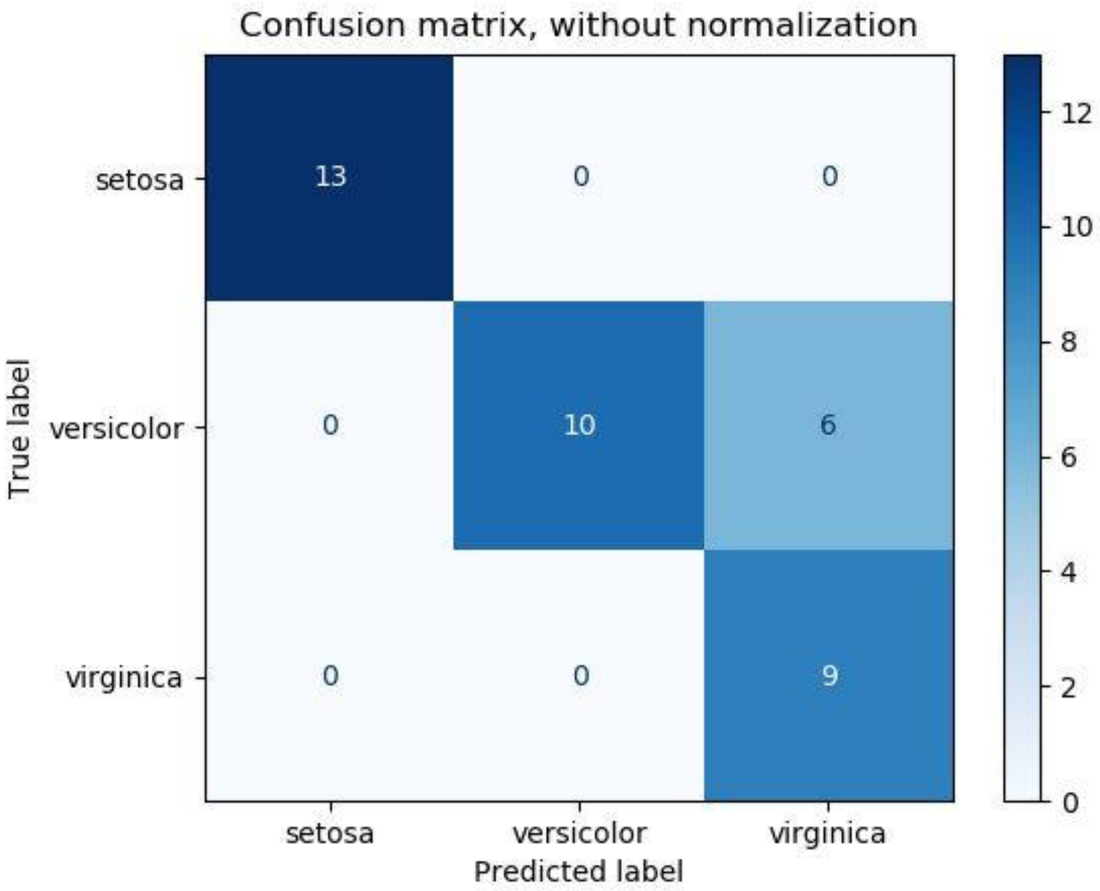
Type I error
(false positive)

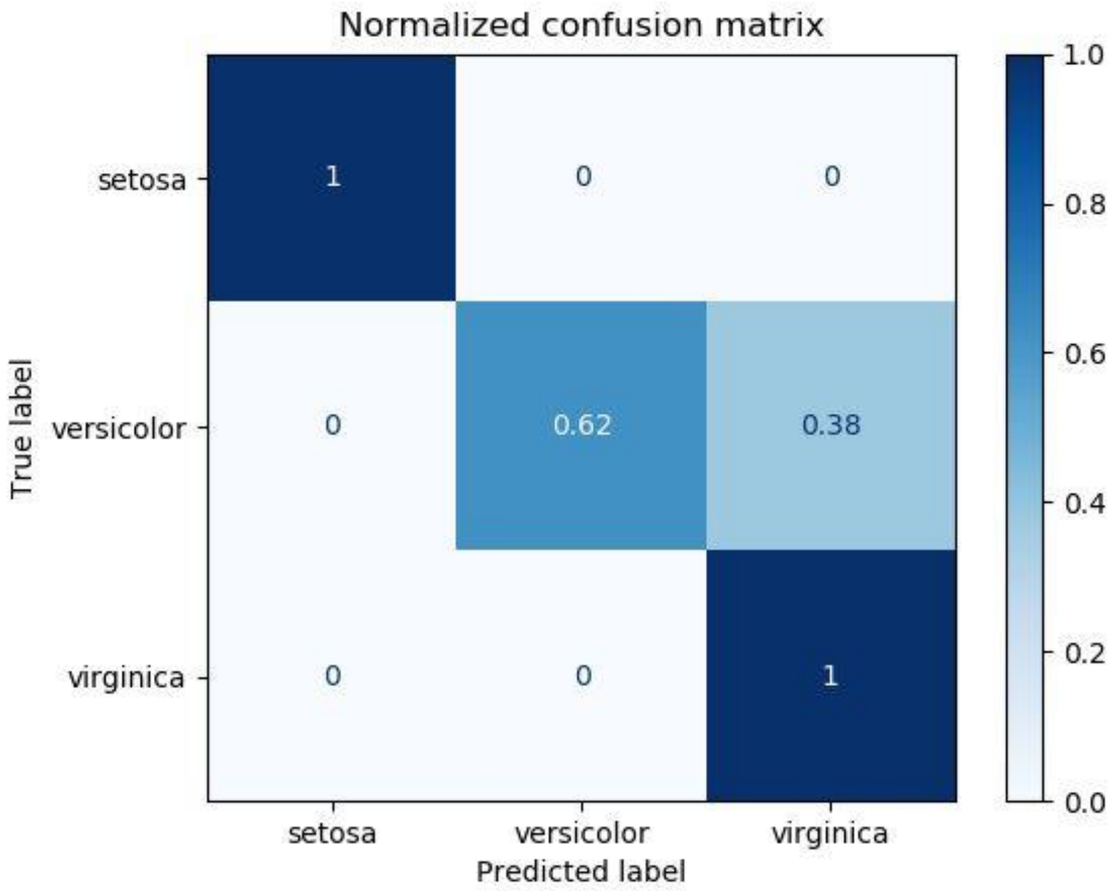


Type II error
(false negative)

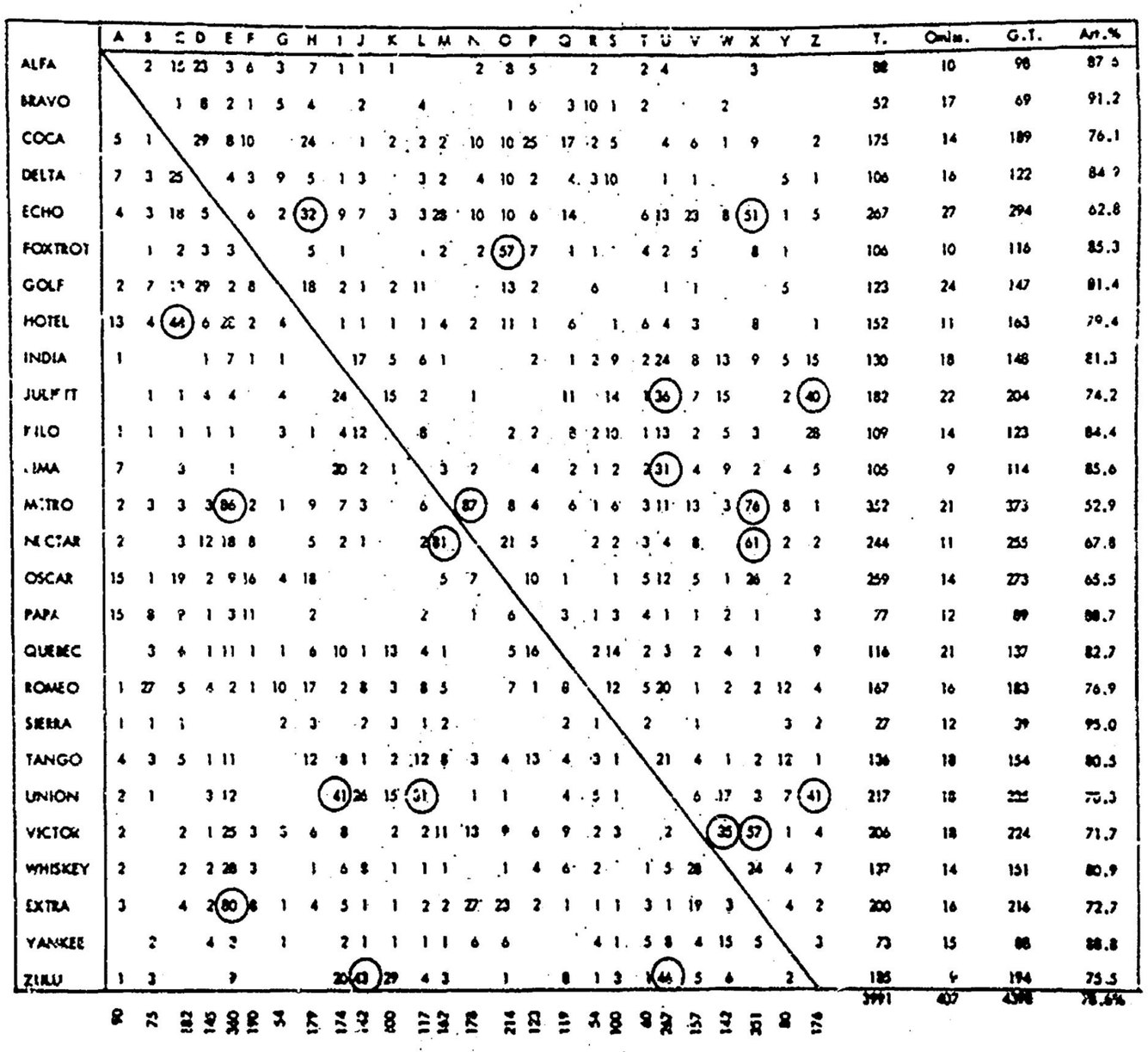


Confusion matrix for multiclass





Confusion matrix analysis



In 1959 scientists analyzed ICAO Word-Spelling Alphabet (Alfa, Bravo, Charlie, etc.) on collisions during pronunciations and distinguished words needed corrections using confusion matrix.

The Evolution and Rationale of the ICAO (International Civil Aviation Organization) Word-Spelling Alphabet, July 1959

Youtube story

Confusion matrix analysis

In Imagenet dataset most correlated classes are chihuahua and cupcake.

Such outcome is hard to predict beforehand, but it clearly appears on multiclass confusion matrix.



Precision and Recall

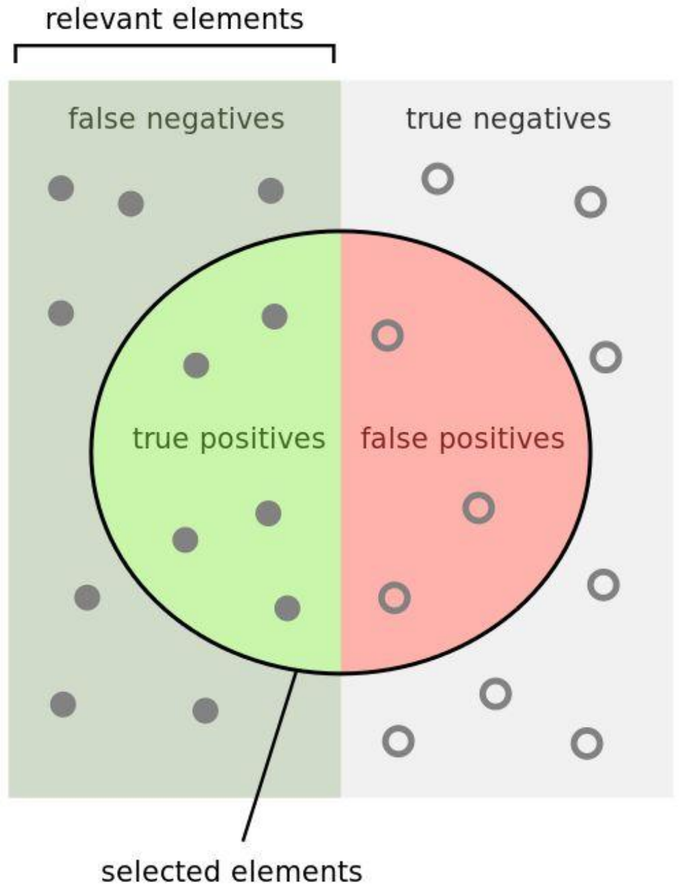
		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**

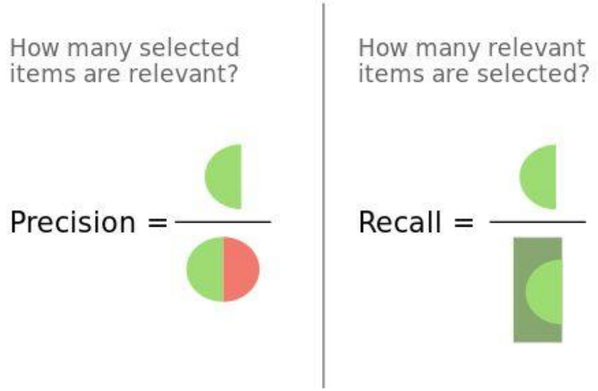
$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Precision and Recall semantics

Precision: P(+|C+) - the probability that the sample is positive, provided the classifier result was positive.

Recall: P(C+|+) - the probability that the classifier result will be positive, provided the sample is positive.





Demonic rule

How can I remember those formulas for Precision and Recall?



$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Mnemonic rule:

For **P**recision use F**P** in the formula.

And for remaining metric (Recall) use remaining error (FN).

How to assign labels?

What value will Precision have for simplest baseline if dataset consists in 99% of positive objects?

Can we do something with that?

Yes, reverse labels!

		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

Maximize them!

Construct a model which maximizes:

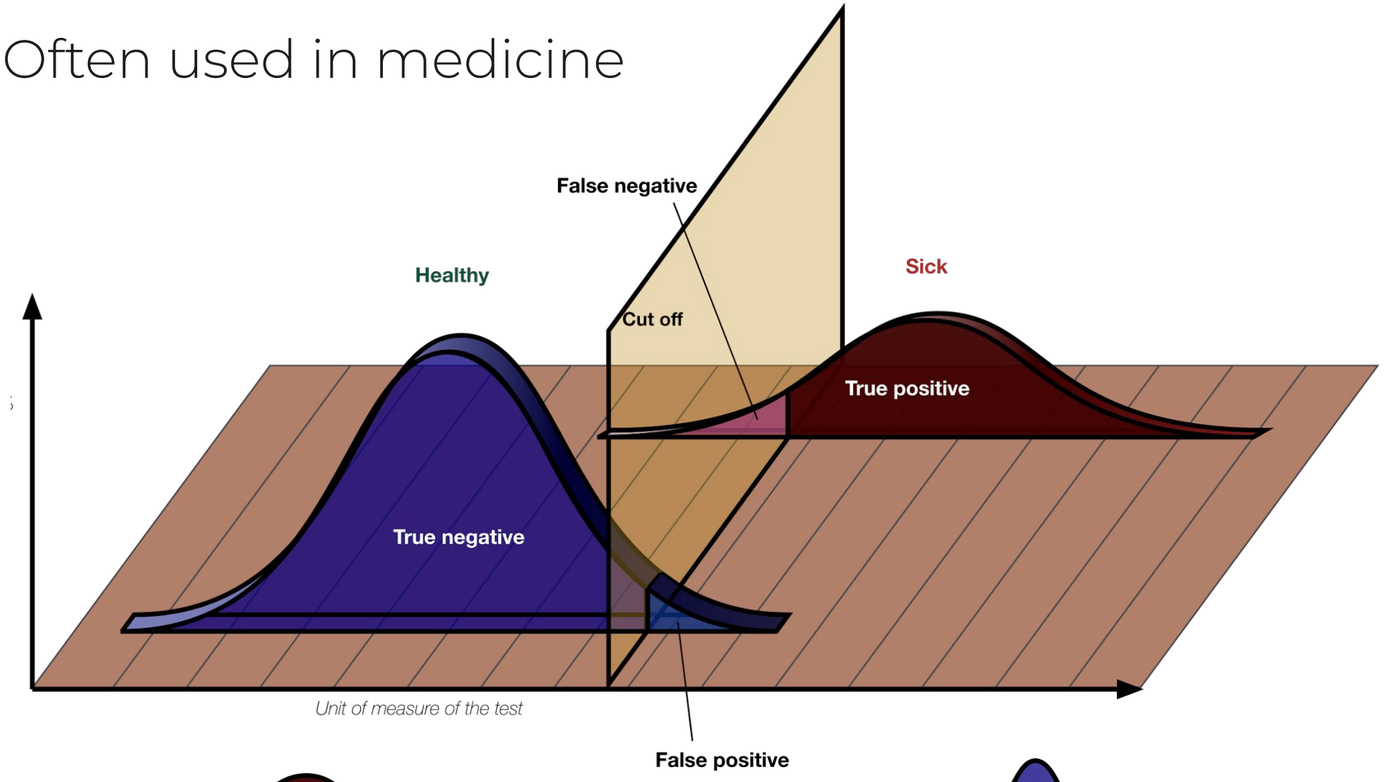
- Recall
- Precision

		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**

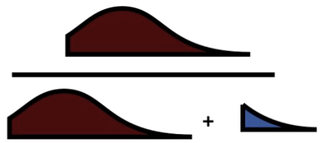
$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Sensitivity and specificity

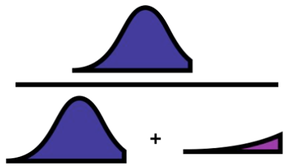
Often used in medicine



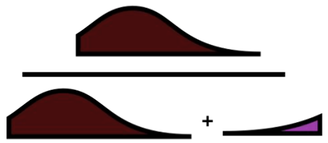
PPV =



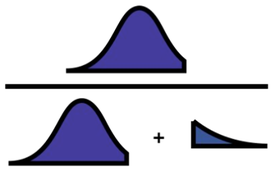
NPV =

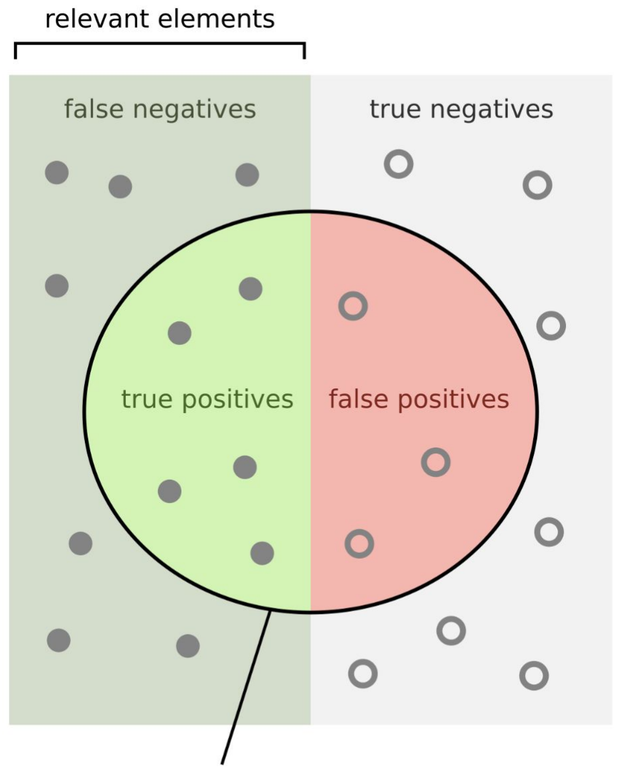


Sensitivity =



Specificity =





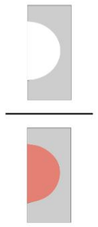
How many relevant items are selected? e.g. How many sick people are correctly identified as having the condition.

How many negative selected elements are truly negative? e.g. How many healthy people are identified as not having the condition.

Sensitivity=



Specificity =



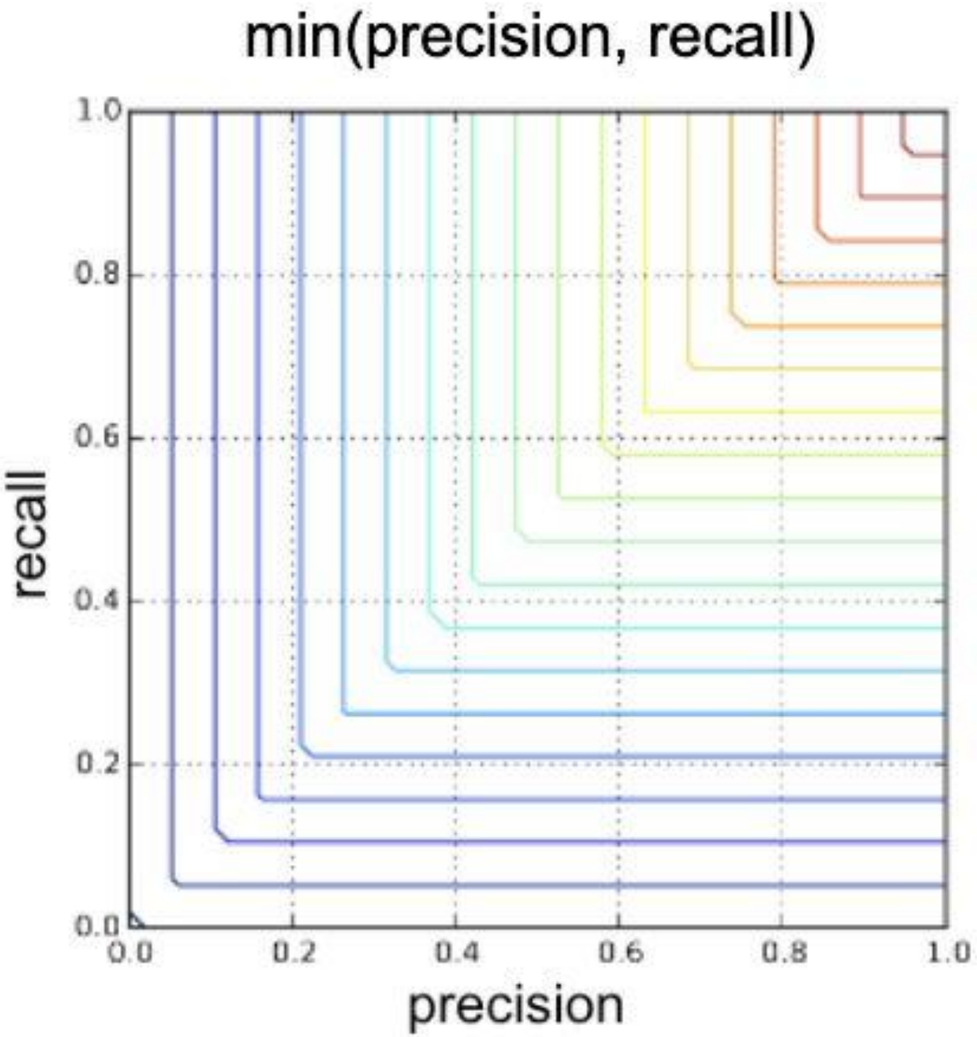
Plenty of other ratios

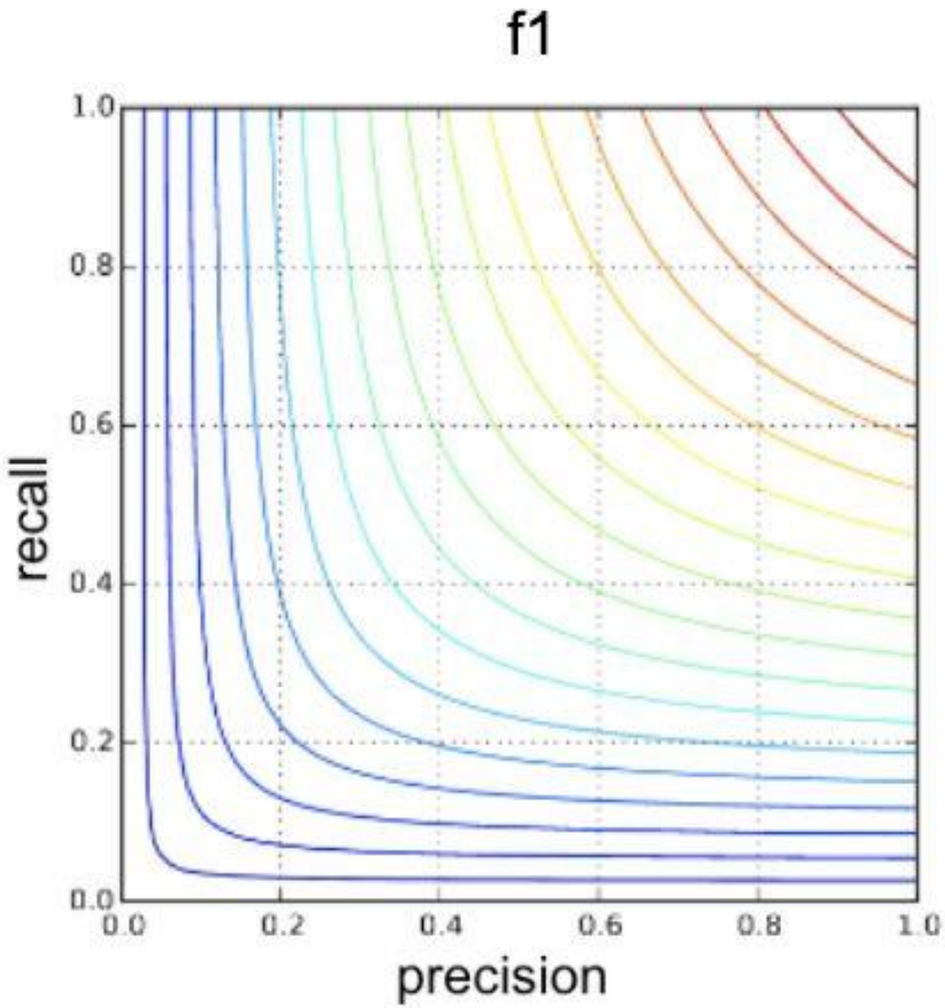
		Predicted condition		Sources: [12][13][14][15][16][17][18][19] view · talk · edit	
	Total population $= P + N$	**Predicted positive (PP)**	**Predicted negative (PN)**	Informedness, bookmaker informedness (BM) $= TPR + TNR - 1$	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	**Positive (P)** [a]	**True positive** (TP), hit[b]	**False negative** (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate type II error [c] $= \frac{FN}{P} = 1 - TPR$
	Negative (N)[d]	**False positive** (FP), false alarm, overestimation	**True negative** (TN), correct rejection[e]	False positive rate (FPR), probability of false alarm, fall-out type I error [f] $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR−) $= \frac{FNR}{TNR}$
	Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) $= \frac{TN}{PN} = 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$
	Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2\,PPV \times TPR}{PPV + TPR} = \frac{2\,TP}{2\,TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$

Different disciplines invent their ratios to fit their problem setup

F-score motivation

We need the one metric to rule them all!





F-score

Harmonic mean of precision and recall

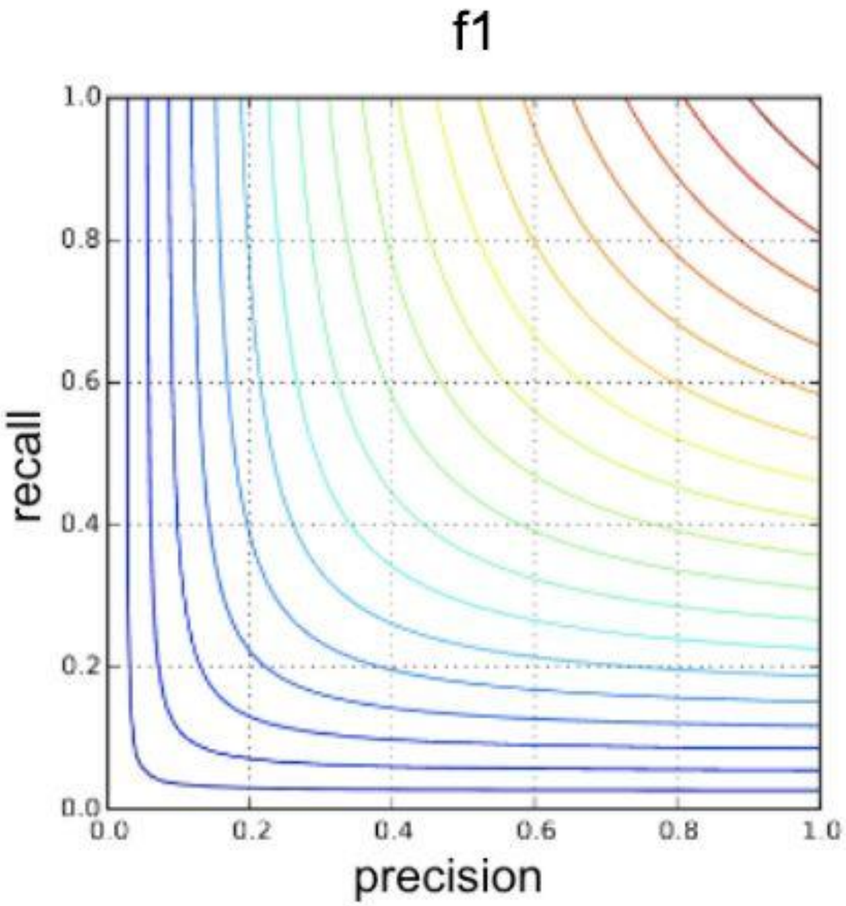
Tends to be closer to the smaller one

$$F_1 = \frac{2}{\mathrm{precision}^{-1} + \mathrm{recall}^{-1}} = 2\,\frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

Generalization to different ratio between

Precision and Recall

$$F_\beta = (1 + \beta^2)\,\frac{\mathrm{precision} \cdot \mathrm{recall}}{\beta^2\,\mathrm{precision} + \mathrm{recall}}$$



Effort to symmetrize F-score is P4 metric using 4 normalized error rates

Examples of imbalanced errors

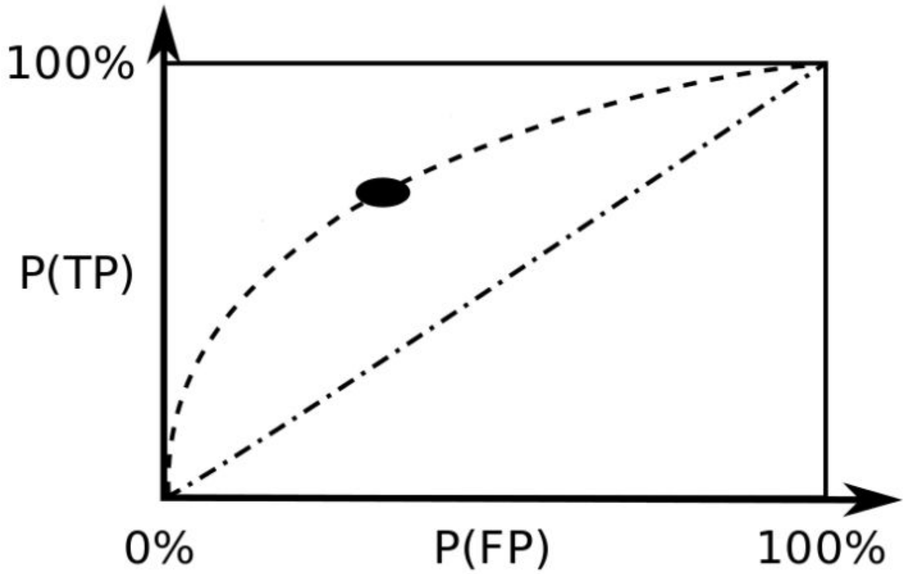


- Medicine
  - COVID test
- Security
  - Flood prediction (and other disasters)
- Banking and finance
  - Loans, mortgages

Equalization coefficient is estimated by risks of FP vs FN

Receiver Operating Characteristic (ROC)

		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**



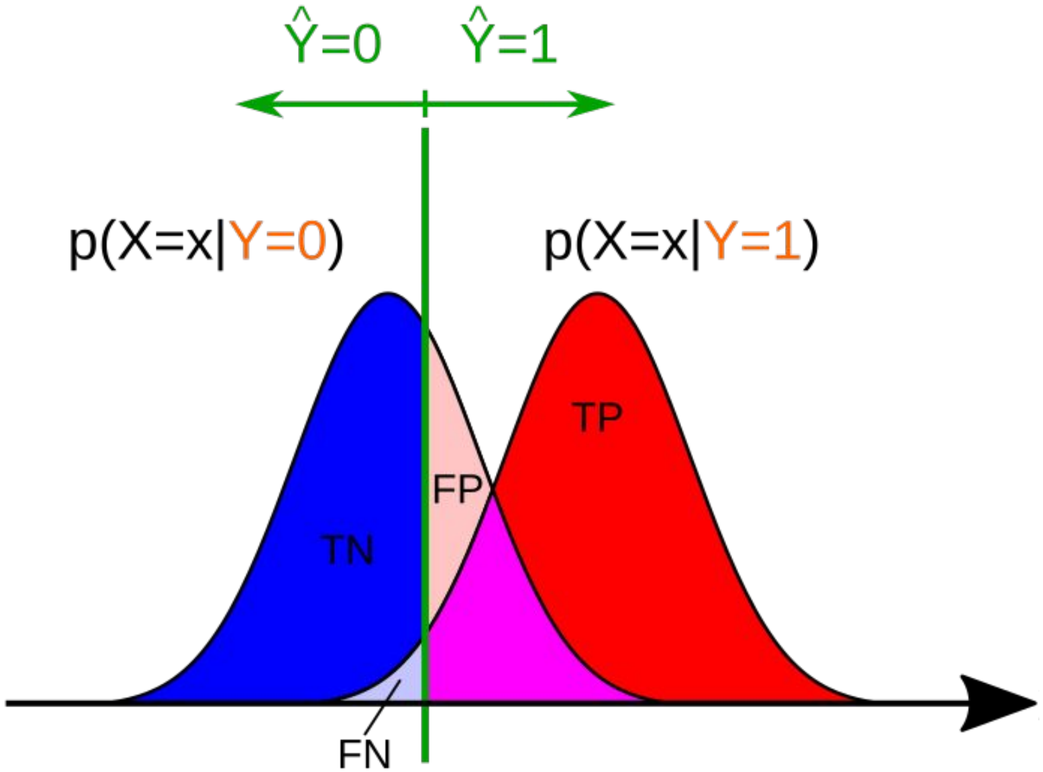
$$\mathrm{FPR} = \frac{FP}{FP + TN}$$

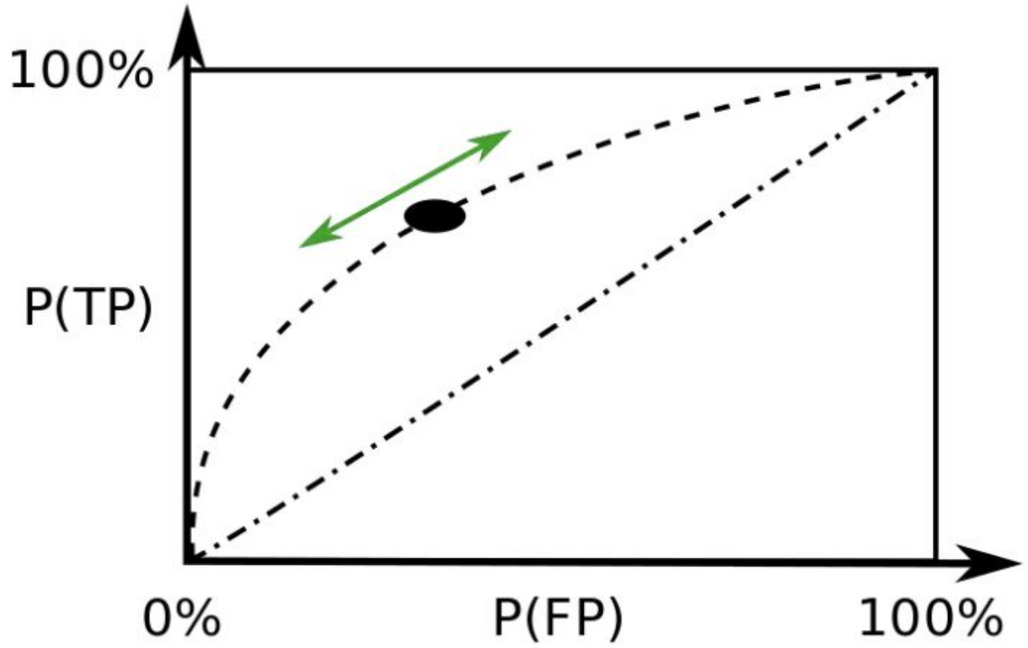
$$\mathrm{TPR} = \frac{TP}{TP + FN} (= \mathrm{Recall})$$

ROC calculation

Classifier have to predict probabilities

Objects get sorted by positive probability

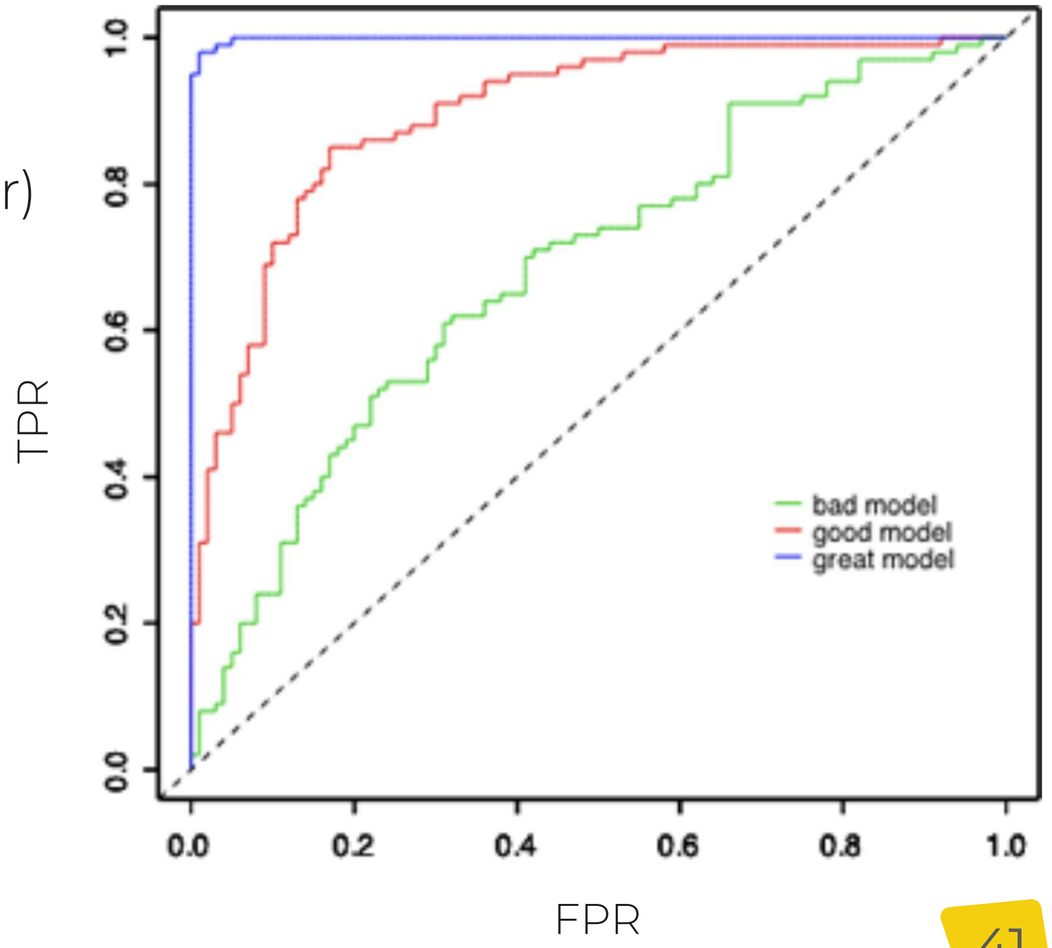




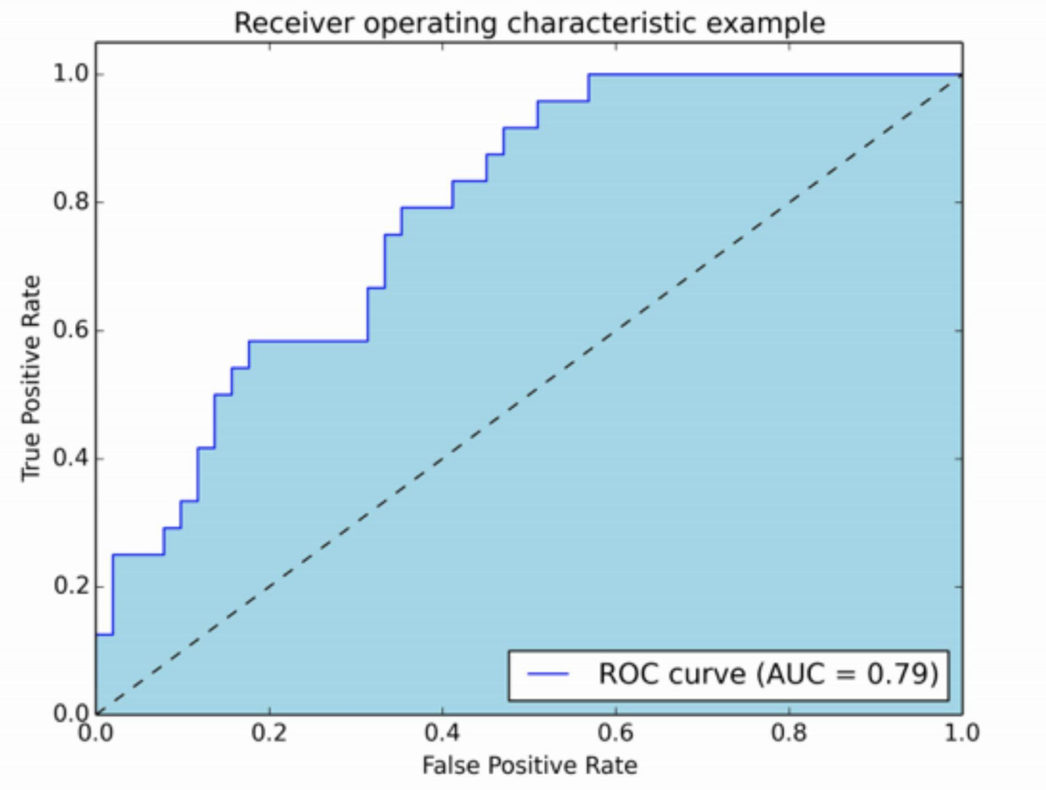
Line is plotted as threshold moves

ROC properties

- It is monotonic due to TPR and FPR
- Baseline is random prediction
- Always above diagonal (for reasonable classifier)
  - If below - change sign of predictions
- Strictly higher curve means better classifier
- Number of stairs (thresholds) proportional to dataset size
- In practice thresholds are fixed before calculation



ROC Area Under Curve (ROC-AUC)



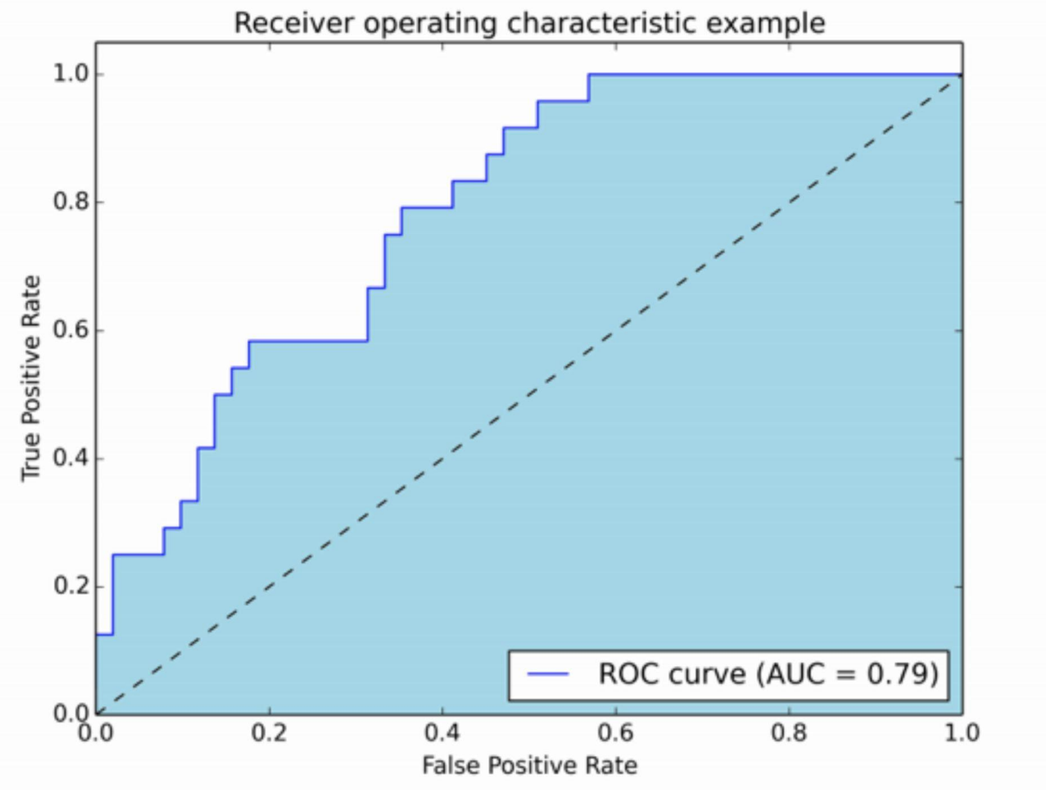
Effectively lays in (0.5, 1)

Bigger ROC-AUC doesn't imply higher curve everywhere

More explanations with pictures

Detailed explanation [ru]

ROC-AUC properties



Equal to fraction of correctly sorted paris
Because we compute it over predictions sorted by score.

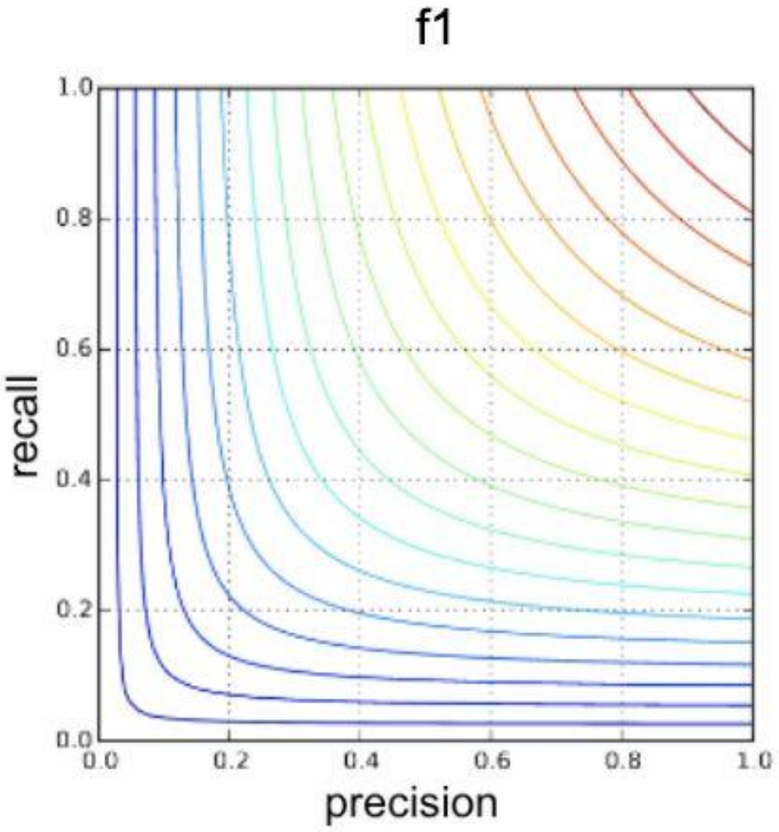
Scale-invariant
It measures how well predictions are ranked, rather than their absolute values.
If we multiply all predictions by constant metric will not change.

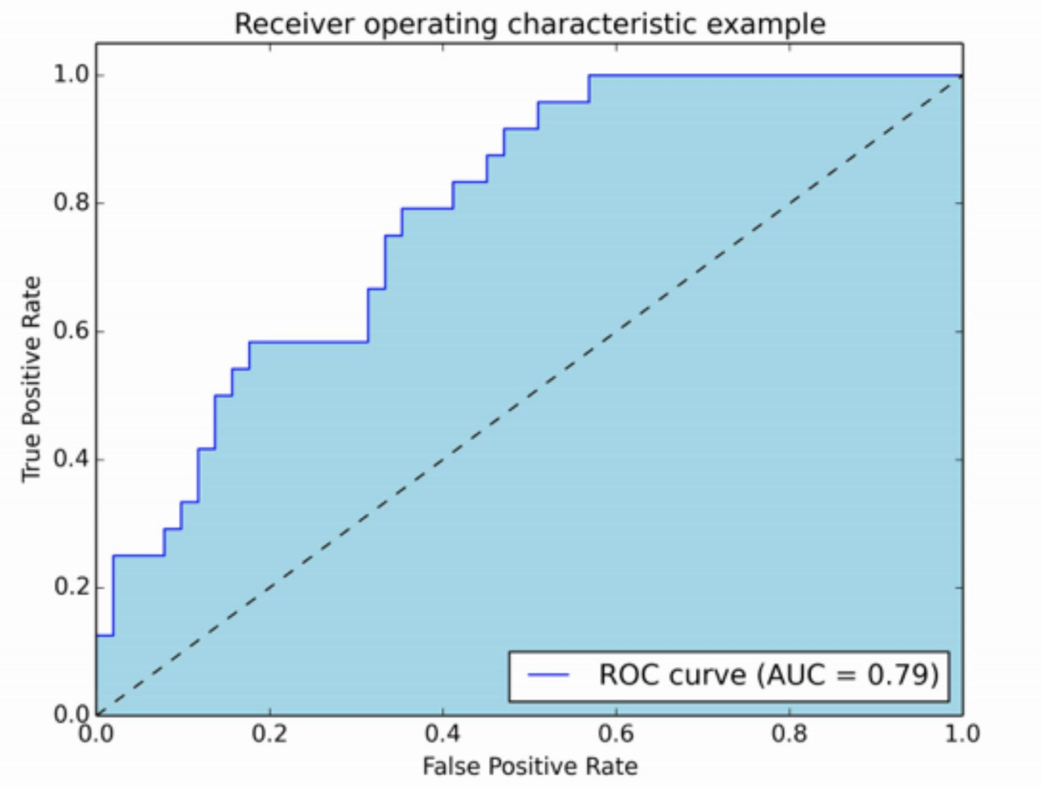
Classification-threshold-invariant
It measures the quality of the model's predictions irrespective of what classification threshold is chosen.

Source

F-score vs ROC-AUC

Which one to tune?





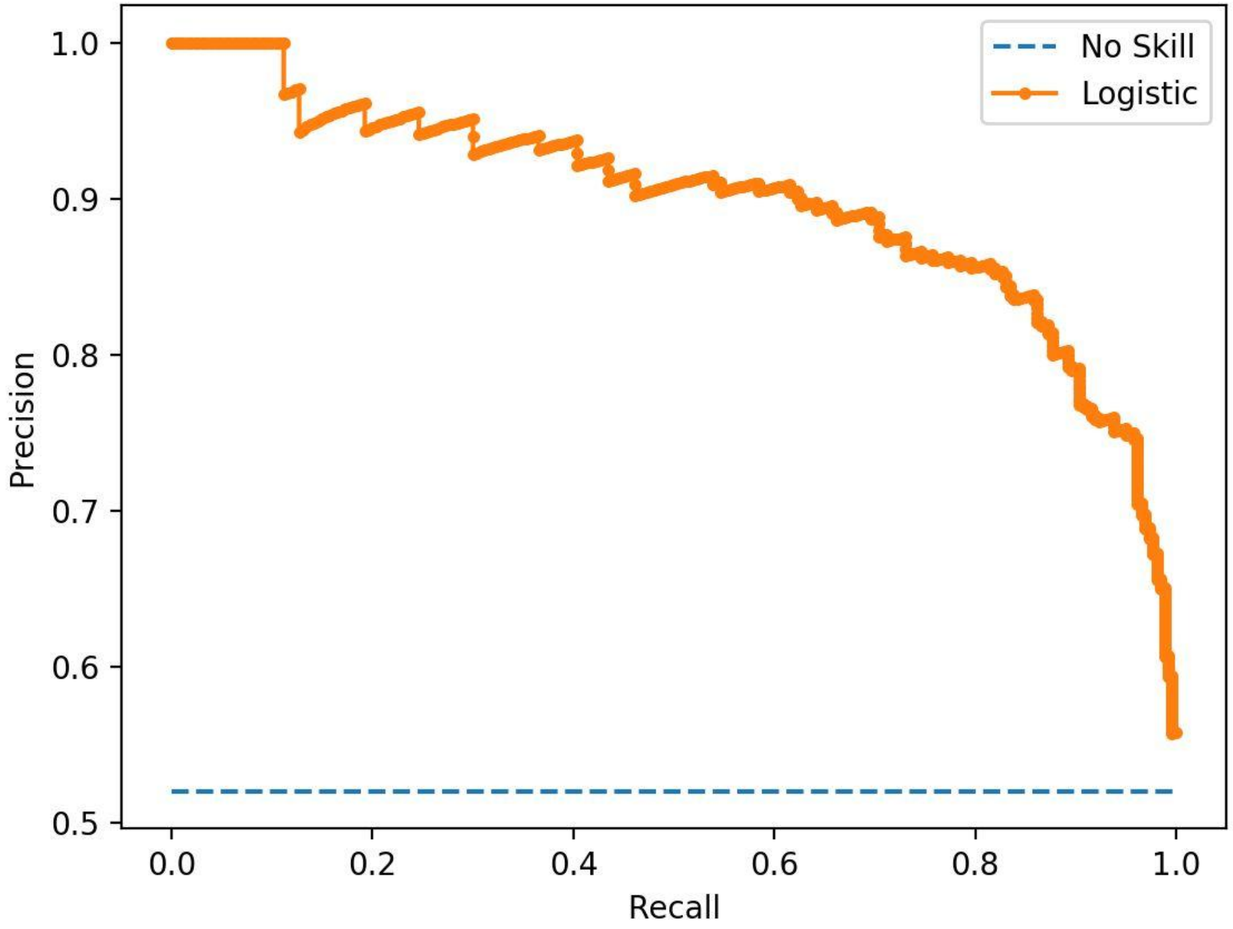
Precision-Recall Curve and AUC

AUC is in (0, 1)

Source of AP metric

(important for next semester)

Nice article



Multiclass metrics



As with linear models we need some magic to measure multiclass problems

Basically it's mean of one or another kind

Detailed info here and here

average	Precision	Recall	F_beta
"micro"	$P(y, \hat{y})$	$R(y, \hat{y})$	$F_\beta(y, \hat{y})$
"samples"	$\frac{1}{\lvert S\rvert}\sum_{s\in S} P(y_s, \hat{y}_s)$	$\frac{1}{\lvert S\rvert}\sum_{s\in S} R(y_s, \hat{y}_s)$	$\frac{1}{\lvert S\rvert}\sum_{s\in S} F_\beta(y_s, \hat{y}_s)$
"macro"	$\frac{1}{\lvert L\rvert}\sum_{l\in L} P(y_l, \hat{y}_l)$	$\frac{1}{\lvert L\rvert}\sum_{l\in L} R(y_l, \hat{y}_l)$	$\frac{1}{\lvert L\rvert}\sum_{l\in L} F_\beta(y_l, \hat{y}_l)$
"weighted"	$\frac{1}{\sum_{l\in L}\lvert\hat{y}_l\rvert}\sum_{l\in L}\lvert\hat{y}_l\rvert P(y_l, \hat{y}_l)$	$\frac{1}{\sum_{l\in L}\lvert\hat{y}_l\rvert}\sum_{l\in L}\lvert\hat{y}_l\rvert R(y_l, \hat{y}_l)$	$\frac{1}{\sum_{l\in L}\lvert\hat{y}_l\rvert}\sum_{l\in L}\lvert\hat{y}_l\rvert F_\beta(y_l, \hat{y}_l)$

Multiclass metrics



- **macro** simply calculates the mean of the binary metrics, giving equal weight to each class
- **micro** gives each sample-class pair an equal contribution to the overall metric (except as a result of sample-weight)
- **weighted** accounts for class imbalance by computing the average of binary metrics in which each class's score is weighted by its presence in the true data sample
- **samples** applies only to multilabel problems

Binary metrics are calculated by One vs Rest scheme

Class imbalance

girafe ai

05

Class imbalance fixing

- Resample data
  - Even for regression
- Object weight in loss function

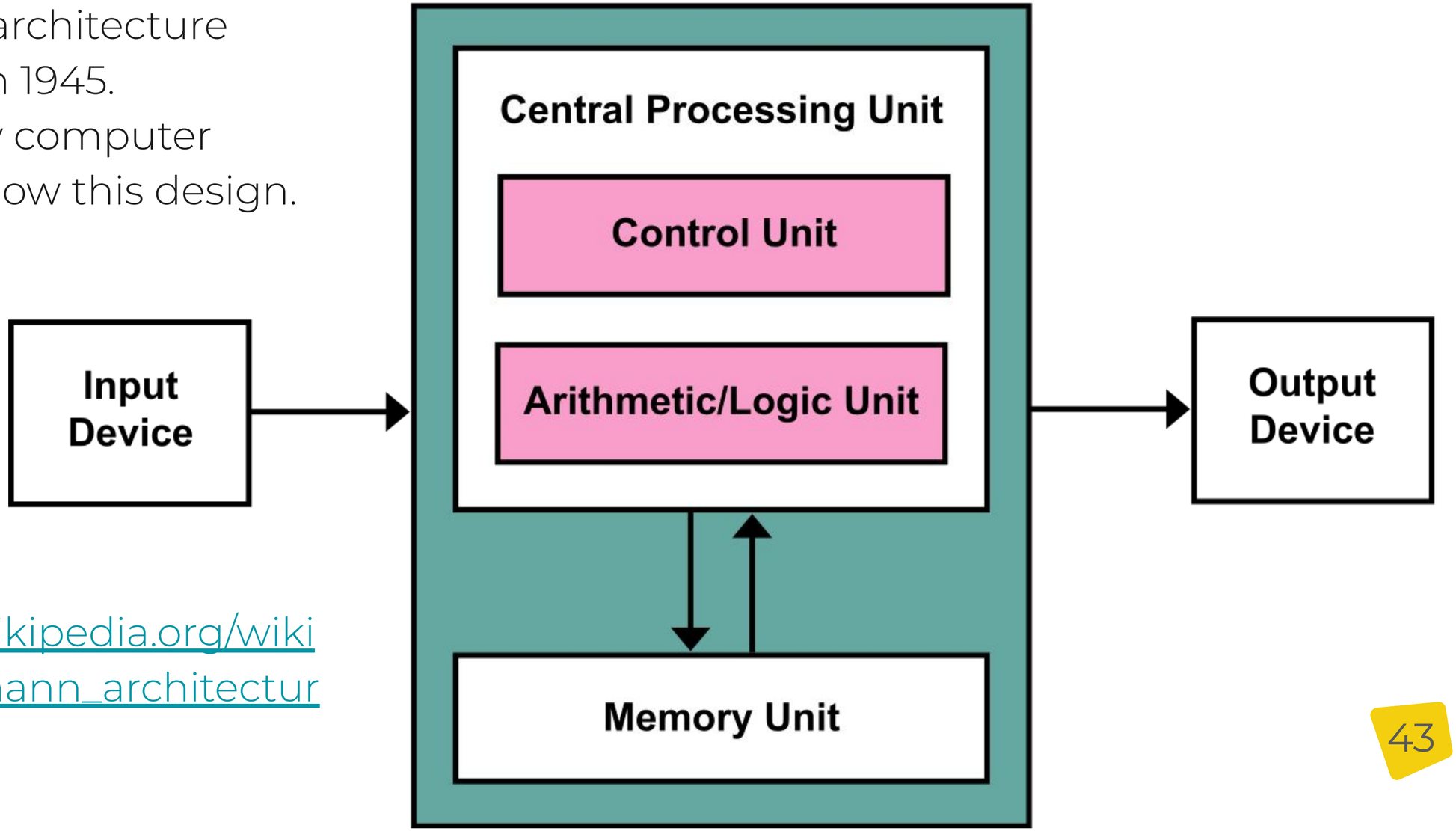
GPU beasts
and where to find them

girafe
ai

06

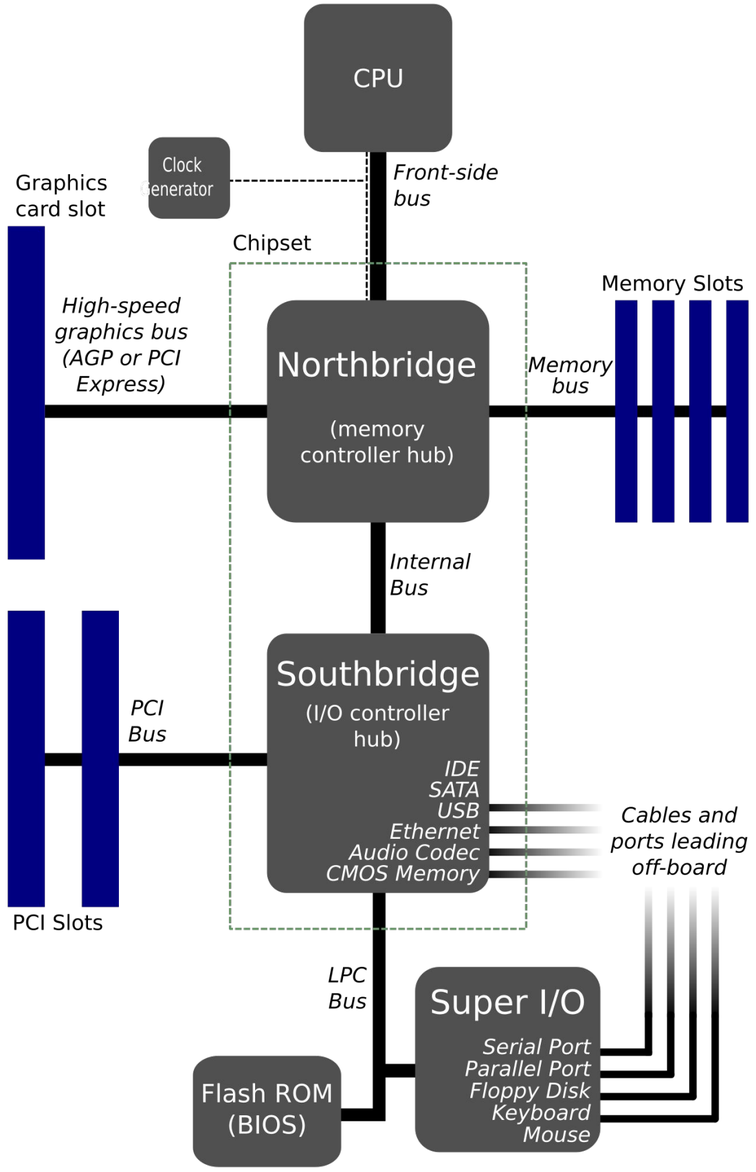
von Neumann architecture

Computer architecture proposed in 1945.
Up to today computer systems follow this design.

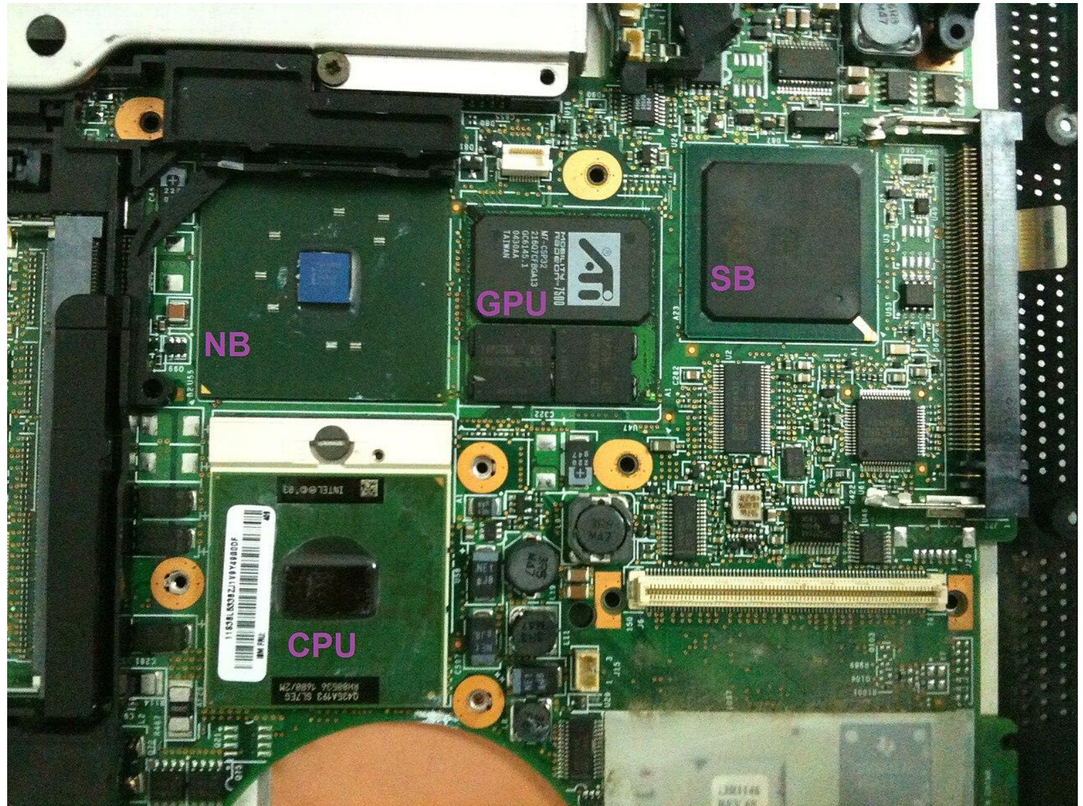


https://en.wikipedia.org/wiki/Von_Neumann_architecture

von Neumann in the wild



e.g. for many years (up to 2010s) this concept was presented in may motherboards by two chips:
Northbridge (CPU, GPU and memory)
Southbridge (all IO: PCI, SATA, Ethernet, USB)



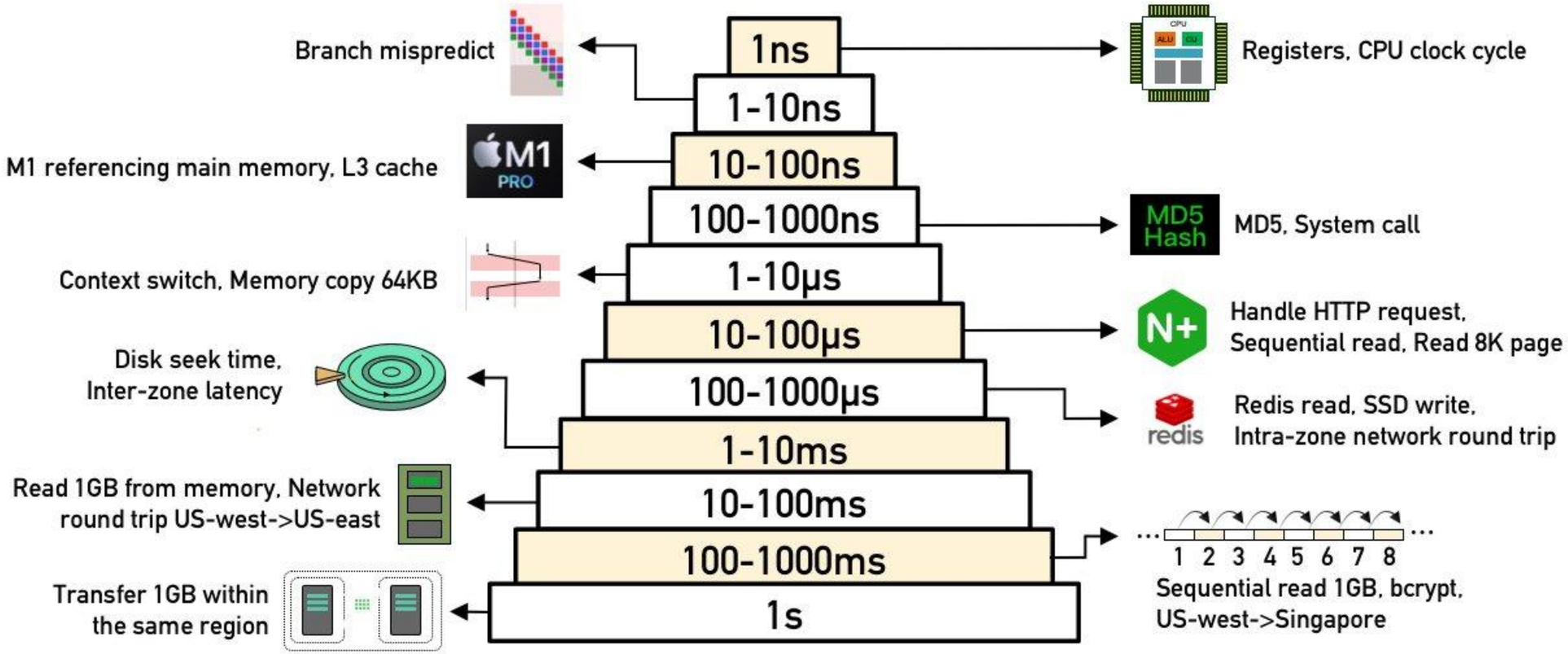
Data Locality principle

[Ulrich Drepper, What every programmer should know about memory (2007)]

[Stackoverflow discussion]

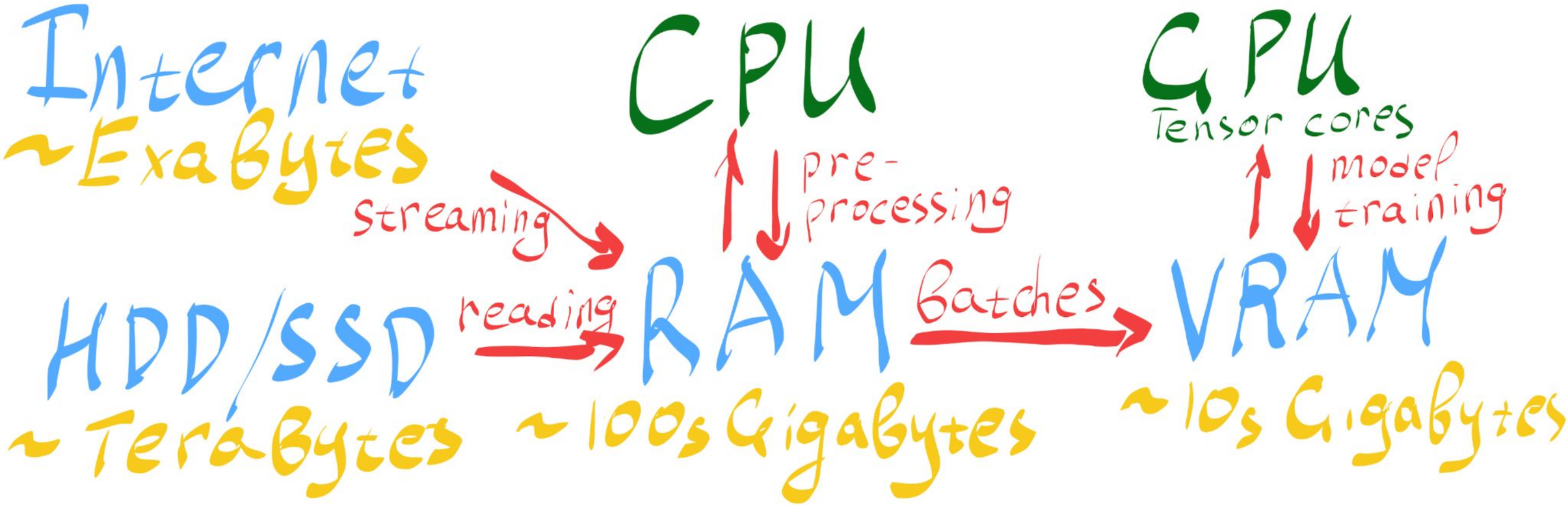
[Locality of reference]

[Memory hierarchy]



Latency for memory operations in 2020s by [bytebytego.com]

Data flow in model training



GPU starvation fulfillment scheme.

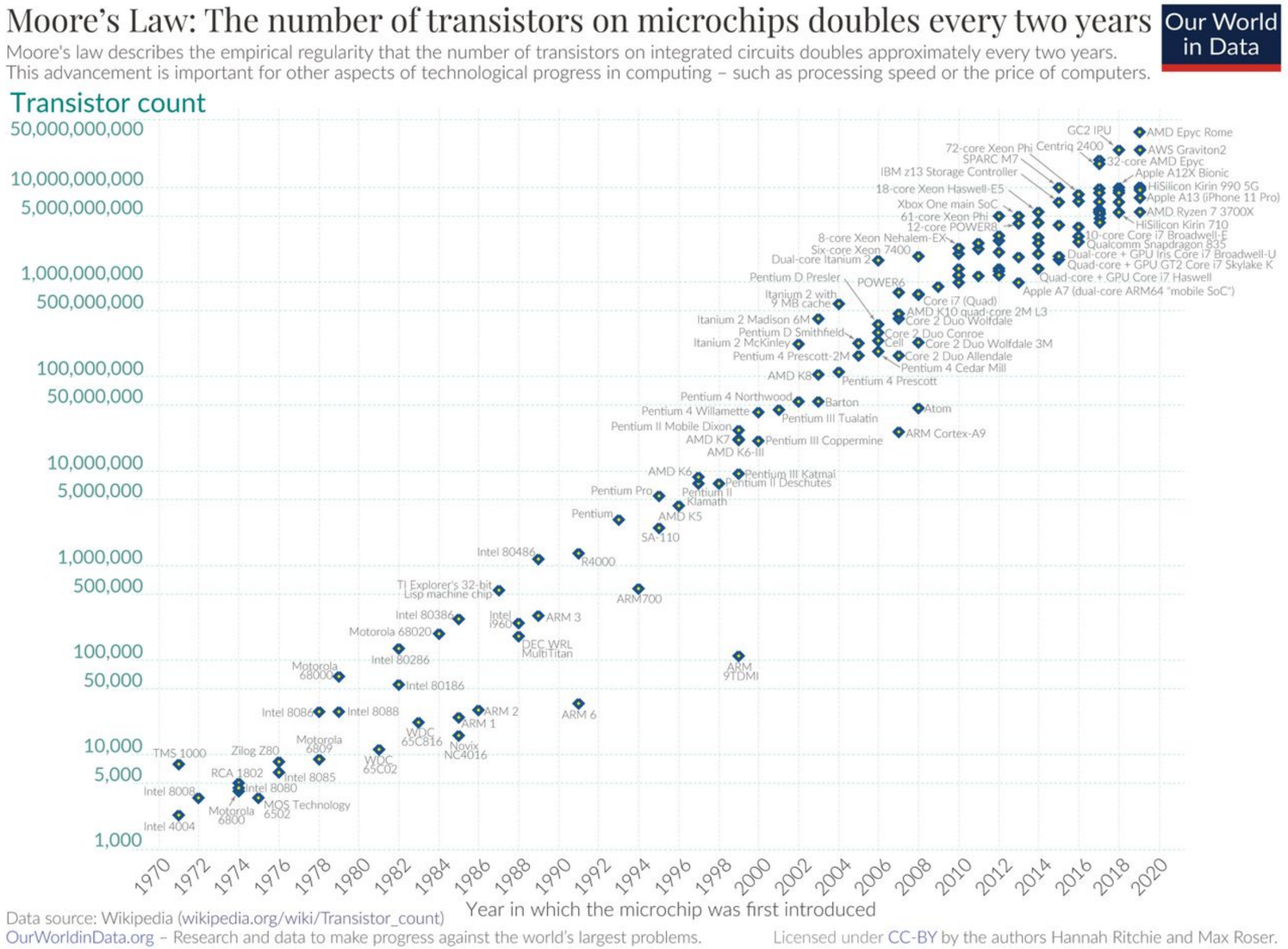
VRAM - video random-access memory. RAM vs VRAM

Moore's law

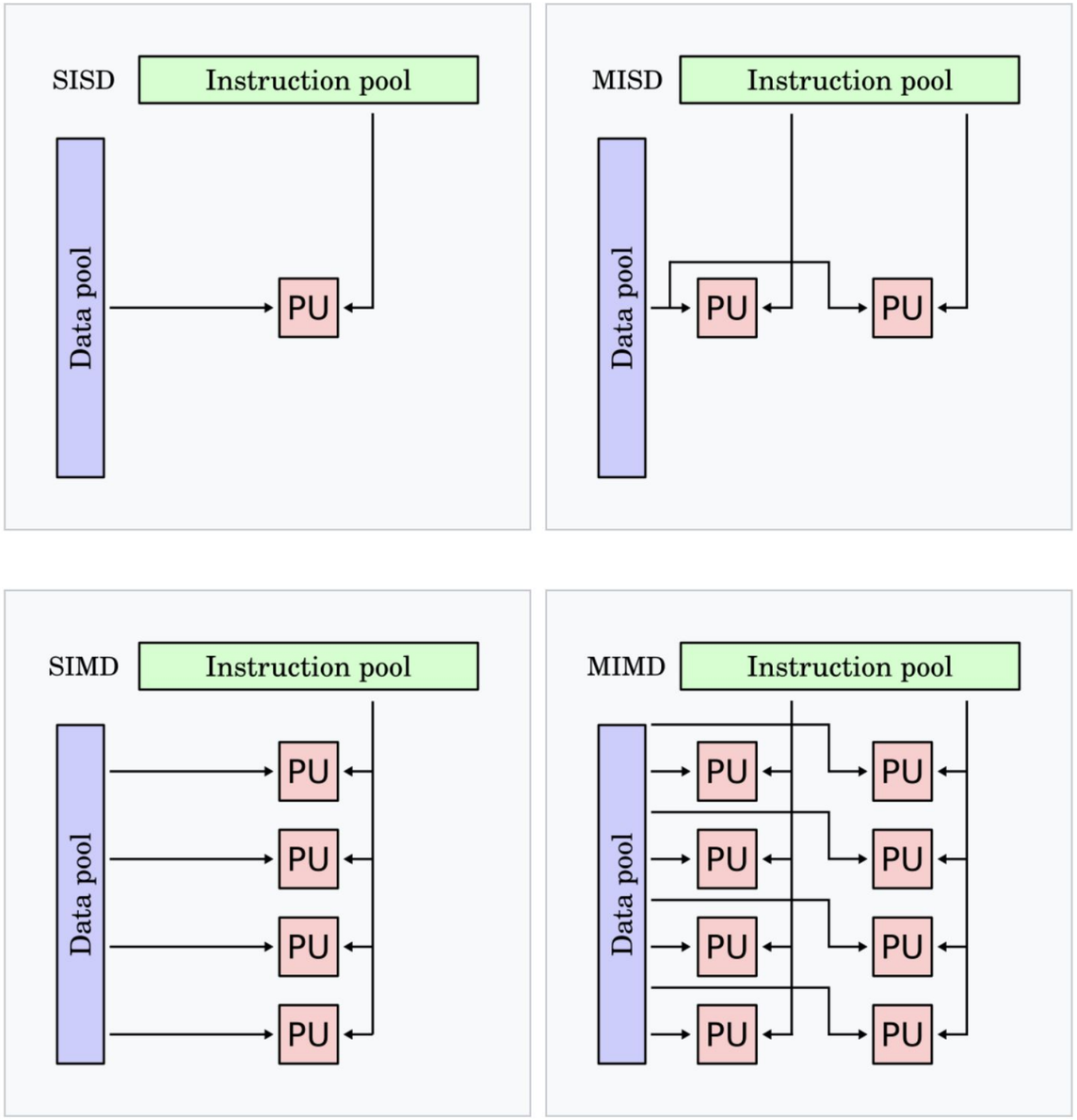
the observation that the number of transistors in an integrated circuit **doubles about every two years**.

Rather than a law of physics, it is an empirical relationship. Nevertheless, it is still carried out to this day.

First noted by Gordon Moore in 1965 in his [Cramming more components onto integrated circuits] article.



Flynn’s taxonomy



classification of multiprocessor computer architectures, proposed by Michael Flynn in 1966.

It has been used as a tool in the design of modern processors and their functionalities.

Further development of this taxonomy leads to pipelined and vector processors.

https://en.wikipedia.org/wiki/Flynn's_taxonomy

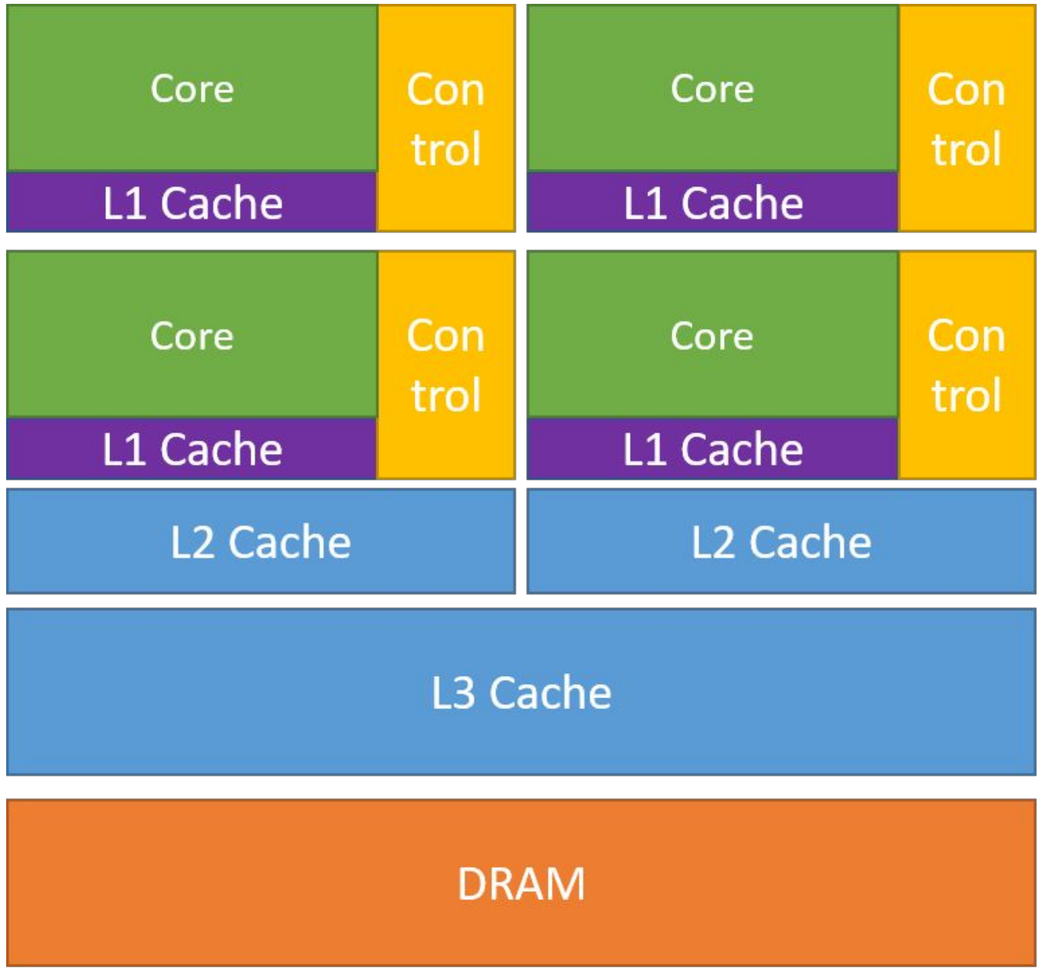
CPU vs GPU metaphor

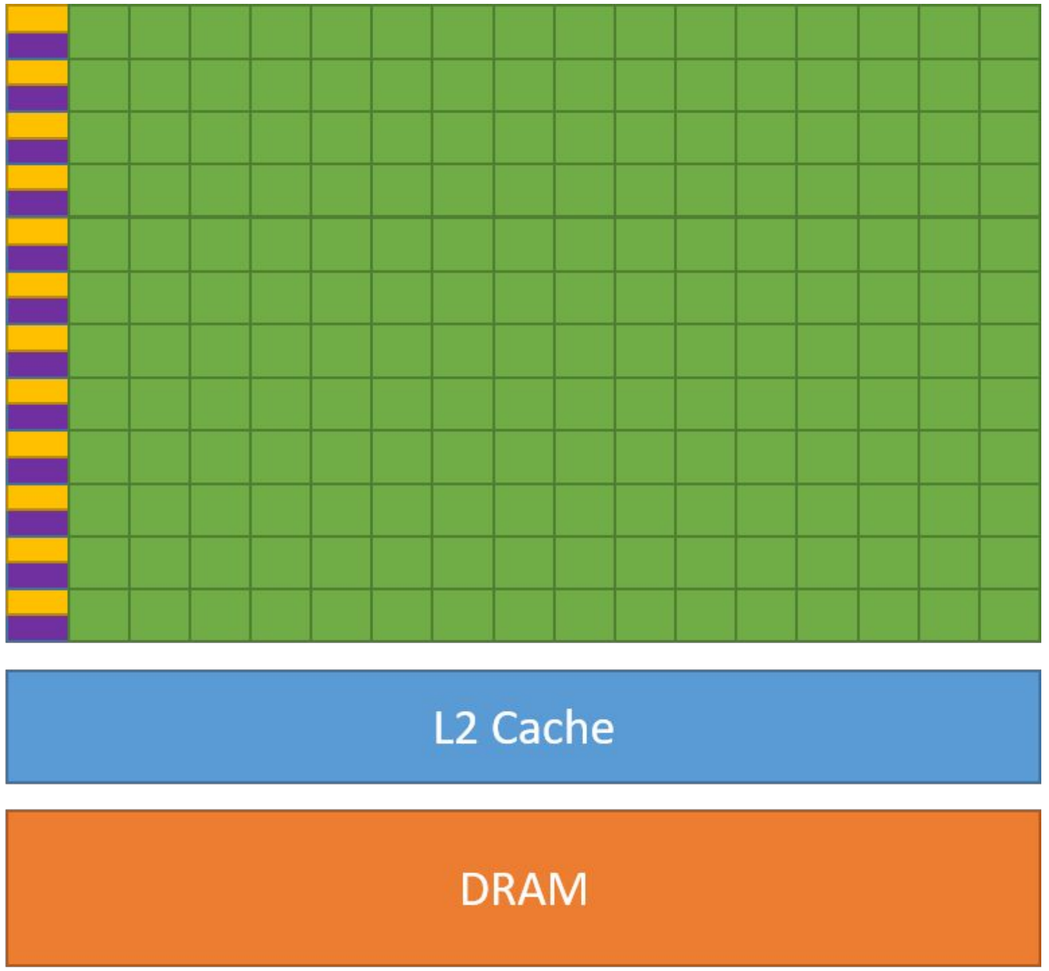




Great conceptual GPU hardware overview
https://www.youtube.com/watch?v=h9Z4oGN89MU

CPU vs GPU architecture





https://docs.nvidia.com/cuda/cuda-c-programming-guide/

Conclusion on CPU vs GPU

A **CPU** spends silicon making a **small number of complicated fast threads**:
big caches, deep control, branch prediction, latency reduction.
CPU often wins for tiny tensors, tiny batches, lots of Python control flow, or workloads dominated by data movement rather than arithmetic.

A **GPU** spends silicon making a **huge number of simpler threads run together**:
many SMs, many arithmetic units, very high bandwidth, and enough on-chip memory to coordinate massively parallel work.
GPU wins when there is enough parallel work to amortize transfer and launch overhead.

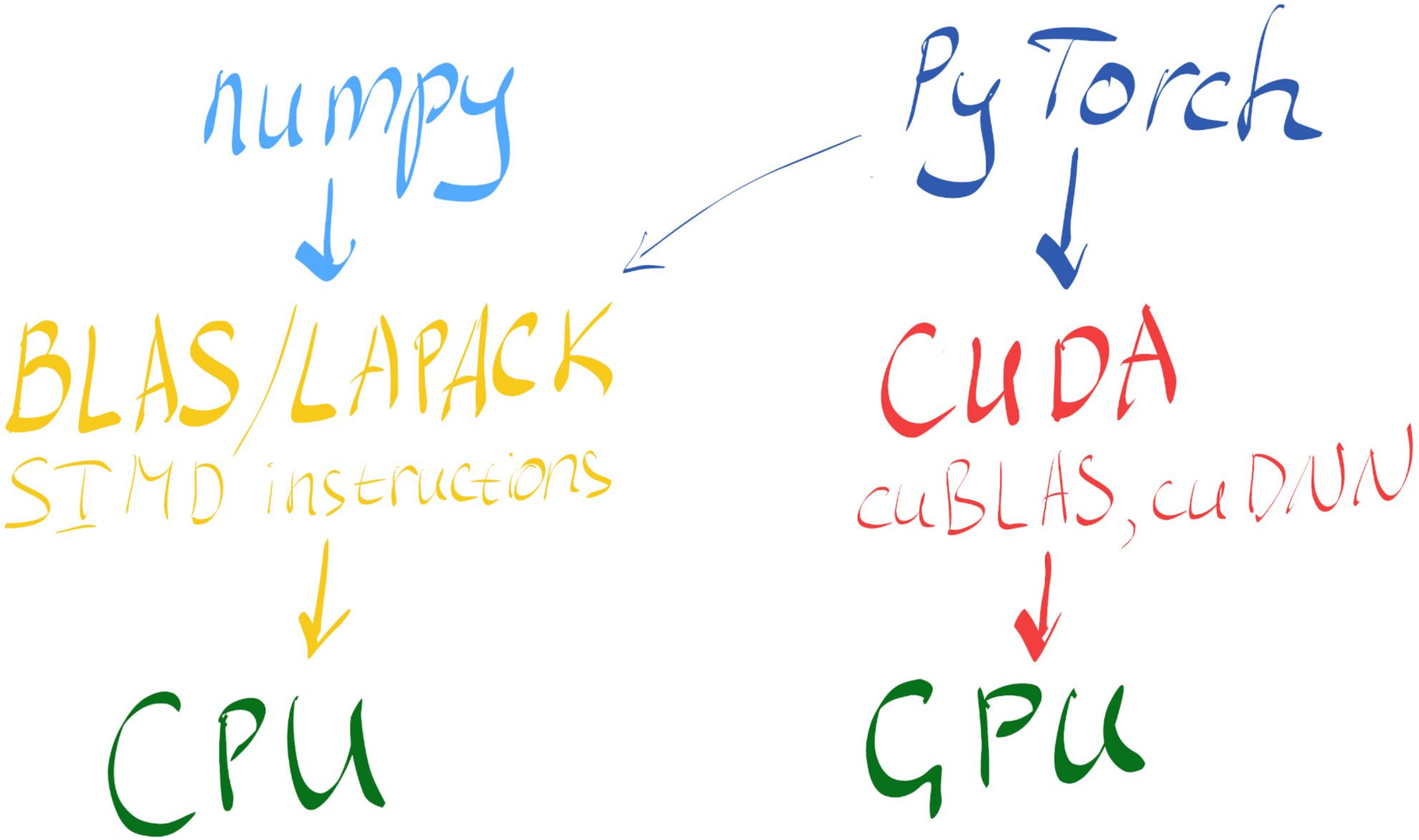
GPU devotes more transistors to data processing, while the CPU devotes more to caching and flow control.

Hardware comparison numbers

EPYC 9965 datasheet	H100 SXM5 datasheet
192 cores	132 Streaming Multiprocessors include: 16,896 CUDA cores 528 Tensor cores
384 hardware threads	270,000 resident CUDA threads
1 KB registers 15 MB L1 cache 192 MB L2 cache 384 MB L3 cache	33,792 KB registers 30 MB L1 cache (shared per SM) 50 MB L2 cache -
up to 6 TB DDR5 RAM	80 GB HBM3
614 GB/s memory bandwidth	3.3 TB/s memory bandwidth

* All numbers are approximate and rudely unified in sake of contrast

Matrix multiplication infrastructure



CUDA ecosystem

Numpy is concentrated on vectorized processing on CPU.
While PyTorch do it on GPU using CUDA.

CUDA which is proprietary parallel computing platform and API that allows to use GPUs for accelerated general-purpose processing.



CUDA API is an extension of the C programming language that adds the ability to specify thread-level parallelism in C and also to specify GPU device specific operations (like moving data between the CPU and the GPU and GPU's virtual instruction set).

https://en.wikipedia.org/wiki/CUDA

Pytorch backends



Main device types in PyTorch:

- cpu
  - like numpy, but slower
- cuda
  - NVIDIA or AMD GPUs
  - they share pytorch's semantics
- mps
  - Apple Silicon GPU cores



torch log

All Images Shopping Videos Short videos Web Forums More Tools

PyTorch
https://pytorch.org › docs › stable › generated › torch.l...

torch.log — PyTorch 2.6 documentation

torch.log. torch.log(input, *, out=None) → Tensor. Returns a new tensor with the natural logarithm of the elements of input .

Images



How to Make a Simple O...
Amateur Camper Ca...



5 Reasons A Swedish To...
Lekto Woodfuels



Swedish Torch Log with ...
YouTube

Show more images

Where to find them



Free GPUs are available on:

- Google colab
- Kaggle

Also you can rent them in various GPU cloud providers or buy yourself one if you want to develop intensively.

Colab subscription overview

Other ways to run jupyter notebooks



- Binder https://mybinder.org/
  - light cloud notebook run
- Jupyter notebook local server
- Jupyter hub https://jupyter.org/hub
  - multitenant server
- Jupyter lite https://jupyterlite.readthedocs.io/en/stable/
  - in-browser jupyter notebook on WebAssembly

Revise



1. Linear classification
    a. margin
    b. loss functions
2. Logistic regression
    a. sigmoid derivation
    b. Maximum Likelihood Estimation
    c. Logistic loss
    d. probability calibration
3. Multiclass aggregation strategies
    a. One vs Rest
    b. One vs One
4. Metrics in classification
    a. Accuracy, Balanced accuracy
    b. Precision, Recall, F-score
    c. ROC curve, PR curve, AUC
    d. Confusion matrix
5. Hardware overview

Next time



- Support Vector Machines
- Principal Component Analysis
- Linear Discriminant Analysis

Thanks for attention!

Questions?



